



THE UNIVERSITY OF HONG KONG
DEPARTMENT OF MECHANICAL ENGINEERING

IDAT7100 Capstone Experience
Final Report
2025-2026

Project Title:	Development and integration of a Bionic Hand System Controlled by Muscle Deformation
Student Names and UID:	Lihao ZHOU 3036507016
Degree Programme:	MSc(Eng)(IDT)
Supervisor:	Dr. WEI Yan Prof. LIU Wei
Date:	01/08/2026

Abstract

Traditional surface electromyography (sEMG) equipment is expensive and highly susceptible to interference from environmental noise. In order to develop a low-cost bionic hand, this project has completely abandoned sEMG technology in favour of a 6-channel Hall-effect muscle deformation armband as the primary interface. This mechanical tracking method successfully eliminates electromagnetic interference at source. In terms of mechanical implementation, the system is based on the open-source TetherIA tendon-driven hand. Both the physical assembly and electrical wiring were completed without modifying any of the original CAD models. This strict control over the hardware minimised costs.

Creating and labelling the dataset was another engineering challenge. I built a custom multi-threaded Python graphical user interface (GUI) to handle data acquisition. Initially, I tested a computer vision workflow for automatic labelling. However, to eliminate synchronisation issues, I later replaced it with a highly discrete, key-triggered protocol. During algorithm evaluation, I tested three traditional machine learning classifiers and a lightweight one-dimensional convolutional neural network (1D-CNN). Offline validation showed that all traditional models performed well. The Random Forest (RF) classifier performed best, achieving an accuracy of 99.94%. This extremely high accuracy indicates a problem: the near-perfect RF score was an artefact of overfitting. Overlapping sliding windows led to time-series data leakage during random cross-validation, a phenomenon consistent with existing literature. During live dynamic testing, the static-feature-based RF model failed to handle the sensor's physical baseline drift. To resolve this, a dynamic zero-mean normalisation layer was integrated into the 1D-CNN. This processing step successfully stripped away the physical baseline drift, allowing the network to extract only the relative waveform features. Ultimately, the normalised 1D-CNN recognised all target gestures in real-time. Despite this functional success, overall classification accuracy and system processing latency still require further optimisation.

Acknowledgement

I would like to thank everyone who supported the development of this project. I am especially grateful to my supervisors, Professor LIU Wei and Dr WEI Yan.

Professor LIU provided guidance on the overall research direction and framework. He didn't micromanage, but constantly encouraged me to think independently and cultivate my ability to solve problems independently.

Dr WEI has always been a vital pillar of my support. Building this system was sometimes frustrating, and whenever I encountered technical bottlenecks or felt overwhelmed by stress, her psychological support always gave me the determination to persevere.

Furthermore, I would like to especially thank the company's technical staff, Mr SHEN and Mr MAO. During the physical assembly of the TetherIA dexterous hand, they provided crucial technical support and shared their experience, helping to solve many assembly problems.

Finally, I would like to thank my family and friends for their support throughout the year.

Statement of Contributions

All work and project development in this report were completed independently by the author.

The core engineering and academic contributions of this project are summarised as follows:

- **Hardware Assembly and System Integration:** The author independently completed the component procurement, physical assembly, and electrical wiring for the 7-DOF TetherIA tendon-driven dexterous hand, evaluated the physical limitations of the open-source framework, and compiled a Failure Mode, Effects, and Criticality Analysis (FMECA) for heat buildup and structural interference.
- **Software Interface and Pipeline Development:** Developed a multithreaded Python GUI operating at a 31 Hz sampling rate. The software initially integrated a computer vision pipeline as an automated labelling tool, which was subsequently replaced by a discrete, keystroke-triggered data collection protocol to eliminate synchronisation latency and recognition errors, thereby stabilising the hardware-to-host data flow.
- **Algorithm Model Evaluation and Phenomenon Confirmation:** Three traditional machine learning models and a 1D-CNN model were compared and tested, observing a progressive trend in the accuracy of traditional models; the data leakage and overfitting phenomena of the Random Forest algorithm under overlapping sliding windows were confirmed, and a comparative analysis with relevant literature was conducted.
- **Anti-drift algorithm exploration and real-time testing:** The effectiveness of the 1D-CNN model with a dynamic zero-mean normalisation layer in removing physical baseline drift was verified. Real-time testing achieved recognition of all target gestures, and the current limitations in accuracy and latency were objectively analysed.

Table of Contents

Abstract	2
Acknowledgement	3
Statement of Contributions	4
List of Figures	7
List of Tables.....	9
List of Variables	10
1. Introduction.....	12
1.1. The Evolution of Dexterous Bionic Hands and HMI	12
1.2. The Bottleneck of Low-Fidelity sEMG Acquisition	13
1.3. Proposed Solution and Project Objectives.....	15
2. Background	19
2.1. Evolution and classification of dexterous robotic hands	19
2.2. Data collection units	22
2.3. Ground truth of the dataset	24
3. Methods.....	27
3.1. Bionic hand assembly.....	27
3.2. Signal acquisition system development.....	31
3.3. Software architecture and GUI	35
3.4. Model architectures and offline data pipeline	40
4. Results and Discussion	44
4.1. Physical hardware evaluation and failure mode validation.....	44
4.2. Comparative analysis of signal acquisition hardware iterations	46
4.3. Feature space distribution and PCA analysis	47
4.4. Offline algorithmic performance and the "Overfitting Illusion"	50

4.5. Real-time dynamic performance, failure mode analysis, and safety-critical decisions.....	52
5. Conclusions and Future Work.....	54
Appendix A	60
Appendix B	64
Appendix C	68

List of Figures

Figure 1. Demonstration of tendon-driven finger [31]	22
Figure 2. Demonstration of Linkage/gear-Driven finger [32]	22
Figure 3. A typical multi-camera RGB-D capture rig designed for high-fidelity, markerless hand kinematic tracking [38]	25
Figure 4. High-fidelity commercial motion capture gloves (Manus Quantum Metagloves) representing a high financial barrier for open-source bionic research [39]	25
Figure 5. Real-time multi-hand tracking [44]	25
Figure 6. Real-time hand tracking with finger distances and timestamp	25
Figure 7. Overview of hand [45]	28
Figure 8. Overview of palm [45]	28
Figure 9. Components and printing parts initially	30
Figure 10. Assembly over servo motors	30
Figure 11. Breadboard setup for AD8232 unit	32
Figure 12. Demonstration of wet electrode patches on forearm	32
Figure 13. Breadboard setup for sEMG sensors	34
Figure 14. Overview of dual-channel sensors over forearm	34
Figure 15. Overview of muscle deformation armband	35
Figure 16. Demonstration of armband over forearm	35
Figure 17. State machine logic and data logging workflow	36
Figure 18. Early version of GUI	38
Figure 19. Final version of GUI	40
Figure 20. Flowchart of the offline preprocessing pipeline and classifier routing	42
Figure 21. Direct-drive rigid linkage of the thumb attached to the FeeTech servo, demonstrating a structural vulnerability to bending stresses	45

Figure 22. High-density integration of seven continuous-duty servo motors within the enclosed palm cavity, highlighting the spatial constraints that lead to thermal accumulation	45
Figure 23. Unnatural signal spikes across all six channels of the magnetic armband, caused by thermal accumulation in the Bluetooth and sensor chips during extended operation.....	47
Figure 24. PCA scatter plots from multiple early datasets. Identical limb movements formed completely isolated subclusters in different datasets, revealing severe signal temporal drift .	49
Figure 25. PCA projection of the final five gesture datasets acquired using a discrete data acquisition protocol. The data points now form tight, cohesive clusters with clear spatial separation	50
Figure 26. This is a boxplot of the normalised distribution of 30 time-domain features extracted from the dataset. No extreme outliers were found, confirming the stability of the baseline data before it was fed into the machine learning model	50
Figure 27. Confusion matrix of the Random Forest classifier evaluated on the full static dataset. The zero off-diagonal values explicitly highlight the overfitting illusion	51
Figure 28. Confusion matrix of the 1D-CNN model evaluated on the 20% hold-out test split, reflecting its true generalisation capacity and realistic classification errors.....	51

List of Tables

Table 1. Comparison between underactuated and fully actuated.....	20
Table 2. Comparison between tendon-driven and linkage/gear-driven	21
Table 3. Technical requirements and bottlenecks of sEMG acquisition architectures.....	23
Table 4. Pin mapping for initial AD8232 sEMG testing configuration	31
Table 5. Pin mapping for dual-channel dry-electrode sEMG configuration.....	32
Table 6. Bill of materials for TetherIA right hand and acquisition units.....	60
Table 7. Failure Mode, Effects, and Criticality Analysis (FMECA) for TetherIA right hand .	62
Table 8. Rating system for FMECA	63

List of Variables

Symbol	Description / Definition	Typical Value / Context
Signal Processing & Windowing		
N	Length of discrete data window (number of sampling points)	$N = 40$ points
S	Step size/stride of the sliding window	$S = 10$ points
C	Number of hardware sensor acquisition channels	$C = 6$ channels
x_i	Discrete amplitude value of the i -th sample within a window	Raw ADC count / mV
$\bar{x}$	Mean amplitude value of a single channel data window	Feature calculation
RMS	Root Mean Square value (signal power indicator)	Time-domain feature
STD	Standard Deviation (signal dispersion indicator)	Time-domain feature
MAV	Mean Absolute Value (average amplitude magnitude)	Time-domain feature
ZCR	Zero-Crossing Rate (frequency domain proxy metric)	Time-domain feature
WL	Waveform Length (cumulative signal waveform complexity)	Time-domain feature
$\mathbb{I}(\cdot)$	Indicator function (returns 1 if condition is true; 0 otherwise)	Mathematical operator
Machine Learning & Model Parameters		
k	Number of folds in Stratified Cross-Validation	$k = 5$
n_{tree}	Number of independent decision trees in Random Forest	$n_{tree} = 100$
Data Analysis & Risk Evaluation		
PC_1, PC_2	First and second Principal Components in PCA feature space	Dimensionality reduction
S	Severity rating of failure impact in FMECA analysis	Scale: 1 – 10

<i>O</i>	Occurrence probability rating of failure in FMECA analysis	Scale: 1 – 10
<i>D</i>	Likelihood rating of failure detection prior to operational stall	Scale: 1 – 10

1. Introduction

1.1. The Evolution of Dexterous Bionic Hands and HMI

The structure of the human hand is extremely complex, and its degrees of freedom are typically defined as 21. If the spatial movement of the wrist joint is also taken into account, the number of degrees of freedom can increase to 27 (i.e. four degrees of freedom for each of the four fingers, five for the thumb, and six for the wrist joint) [1], [2]. Consequently, with further training, humans are capable of performing a wide range of highly complex and demanding tasks, ranging from surgical procedures requiring extremely precise manipulation to lifting heavy objects.

Robotic operation relies primarily on two approaches: mathematical modelling and motion trajectories. Although these early systems performed well in industrial tasks, they lacked the adaptability required to cope with unpredictable environments. With technological advances, modern robots are no longer merely machines that repetitively execute fixed instructions; rather, they are data-driven real-time systems capable of responding to their surroundings [3]. This evolution has had a direct impact on the development of robotic end-effectors. The field is no longer reliant on simple two-finger grippers. Still, it is increasingly focused on highly flexible, real-time feedback to simulate the complex degrees of freedom found in human biology [2], [4], [5]. Now, by using advanced algorithms and combining them with more advanced drive manufacturing processes, these modern bionic hands can achieve adaptive in-hand operation that is difficult to solve with traditional control theory. [6]. Building dependable automated control systems and data collection pipelines for these complex hands remains a major engineering hurdle.

Solving this engineering problem holds real practical value for upper-limb prosthetics. For individuals dealing with limb loss or severe motor impairments, regaining the ability to perform daily tasks relies heavily on the prosthesis's physical capability and precise control [7]. Current

prosthetic or robotic hands still differ significantly from normal human hands in appearance and function, and the cost remains very high. While robotic hands/arms can currently be manipulated, this is primarily done through joysticks or game controllers, which are clearly unsuitable for prosthetics [8], [9]. For amputees, a more intuitive and direct control method is needed to establish a link between awareness and control.

Human-machine interface (HMI) aims to translate human consciousness into executable commands, and several solutions already exist in this field. Among the many methods, surface electromyography (sEMG) has long been considered the primary non-invasive approach for this purpose [10]. When a user contracts a muscle, motor neurons generate electrical action potentials that propagate along muscle fibres and can be recorded on the skin surface. Currently available products use high-density sEMG sensor arrays combined with advanced digital signal processing techniques to decode these signals with low latency [11]. While this high-density approach achieves excellent classification accuracy and intuitive control, it has very stringent requirements for both hardware and software. These requirements make the method expensive and limit its application in basic open-source engineering research.

1.2. The Bottleneck of Low-Fidelity sEMG Acquisition

Despite significant advancements in kinematic performance and control algorithms, a primary limitation of current bionic hands is their cost. This cost is not only limited to the highly complex mechanical design of the bionic hand itself, but also largely stems from the human-machine interface (HMI). High-fidelity surface electromyography (sEMG) controllers require specialised electrodes, advanced signal amplification, and rigorous noise filtering to acquire microvolt-level signals, and this part accounts for a large portion of the total cost. The price of commercial sEMG acquisition systems alone typically exceeds €20,000, which poses a significant obstacle to both research and clinical applications [12], [13]. Therefore, whether as wearable electromyographic prostheses or as remote controllers for robotic manipulation,

complete integrated systems are usually very expensive, often costing between €8,000 and €66,000 or more [14]. For example, commercial models like the BeBionic hand and i-Limb remain unaffordable for most users [15]. Therefore, in addition to exploring more advanced technologies, researchers and engineers often explore low-cost consumer-grade acquisition systems to replace expensive sEMG arrays. One approach is to modify modules originally designed for electrocardiogram (ECG) acquisition, such as the AD8232, to capture muscle activity [16]. While this significantly reduces the economic cost of research, the inherent problems of low-fidelity systems mean that software-optimised signals alone cannot solve the physical bottleneck of acquiring sufficiently low-noise signals.

The energy of standard sEMG signals is mainly distributed over a wide frequency range, typically between 20 and 500 Hz, with the main energy concentrated between 20 and 150 Hz [17]. Achieving independent multi-finger control largely depends on extracting nonlinear, high-frequency motor unit action potentials from this broadband signal [18]. In contrast, low-cost ECG modules are equipped with specially designed, powerful hardware bandpass filters to isolate low-frequency cardiac cycles (typically 0.5 to 40 Hz) and eliminate high-frequency environmental interference. When such hardware is used as a system for acquiring electromyography, the high-frequency details of muscle fibres are filtered out by the hardware, thus simplifying the output to a narrow-band low-frequency signal. This causes certain gestures, particularly those characterised by high-frequency features, to lose their characteristics and become unlearnable by subsequent models. In the initial stage of the signal acquisition system experiment in this project, a single-channel acquisition unit built with the AD8232 ECG module was used to test its signal acquisition characteristics. Empirical tests showed that algorithmic compensation alone could not overcome these hardware limitations. One issue is signal bandwidth compression, and the other is noise. Due to the compression of the signal bandwidth, the gesture characteristics tested were all affected, especially those concentrated in

high-frequency forceful gestures. Even using advanced neural networks, the data resolution required to distinguish subtle muscle intentions was insufficient.

In addition to frequency truncation, this project also developed and built a dual-channel dry-electrode low-fidelity sEMG acquisition device and demonstrated issues with signal stability. Unlike traditional ECG and sEMG devices, dry electrodes do not require conductive gel, making them more convenient to wear. However, they are highly susceptible to environmental noise and fluctuations in skin impedance. For example, during continuous operation, users sweat, and the resulting residue on the skin alters the skin's conductivity. This physical change can immediately cause severe baseline interference to the recorded signal [19].

Signal drift is a more serious problem. After prolonged use, surface electrodes inevitably shift on the skin. Switching between users introduces entirely new anatomical structures. These slight mechanical displacements can significantly alter the baseline shape of the recorded waveforms [20]. Due to this drift, any previously collected training data quickly becomes obsolete. Once the signal baseline shifts, the trained model fails in practical applications. To regain control, users must either constantly perform manual recalibration or completely rebuild the training dataset [21].

Hardware limitations are only part of the problem. Building data-driven human-computer interaction systems also requires synchronised kinematic ‘ground truth’. In order to train supervised machine learning models capable of mapping biological signals to mechanical movements, the system requires high-precision hand kinematic data—such as the exact distance between closed fingers—as target labels. Typically, engineers use expensive optical motion capture systems or complex sensor gloves to acquire this data [22]. These methods present significant economic and technical barriers to low-cost development.

1.3. Proposed Solution and Project Objectives

The primary objective is to develop and test a low-cost bionic hand system within a tight

timeframe. The final design comprises three components: a dexterous robotic hand, a software interface and a signal acquisition unit.

The software acts as a central hub, responsible for collecting raw sensor data, filtering it and matching it with target labels to train a classification model. Once training is complete, the system can operate locally, recognising gestures in real time and converting the user's limb movements directly into motor commands. This work began with the physical end-effector. After evaluating various robotic hand designs and weighing up factors such as finger dexterity, structural complexity and manufacturing costs, the open-source TetherIA tendon-driven dexterous robotic hand was ultimately selected. The structure of the robotic hand remained unchanged. We generated a new bill of materials (BOM), procured all components, and then carried out manual assembly and calibration. Only the right-hand model was constructed for this project.

After the robotic arm was assembled, the research focus shifted to the acquisition of bio signals. Initial hardware testing used a low-cost electrocardiogram (ECG) module; later, it was iterated and upgraded to a dry electrode surface electromyography (sEMG) unit. However, both simulation methods failed due to hardware limitations. The ECG hardware truncated critical high-frequency data, and both circuits were highly sensitive to background electrical noise. Given these shortcomings, the sensing method was changed to track the physical morphology of the muscle. To this end, the researchers purchased a muscle deformation armband equipped with a Hall effect sensor array for testing. Instead of reading current, the device measured the actual physical movement of the muscle. This mechanical approach ignored the impedance problems common in ECG and surface electromyography and was able to easily cope with the effects of environmental electrical noise and skin moisture (such as sweat). However, tracking the physical morphology did not solve all the problems. Displacement of the mechanical sensors still caused spatiotemporal signal drift [23], [24], [25]. Even so, the significant

reduction in background noise made this Hall effect armband the most reliable hardware choice for the project to date.

I developed a custom graphical user interface (GUI) to integrate the hardware. It centrally implements data visualization, label mapping, and real-time model testing. Most importantly, it solves the ground truth problem without requiring expensive motion capture equipment. Early versions of the software used webcams for automatic visual data annotation. However, running visual tracking alongside high-speed data collection caused severe lag and synchronisation errors. To fix this, the camera tracking was replaced by a manual, discrete logging approach. In the final setup, the user selects the target gesture in the GUI and presses the spacebar to start recording. This keystroke method removes the camera entirely. It guarantees that every gesture variation receives an exact timestamp, resulting in a much cleaner dataset. Moving away from early terminal-based scripts to this unified GUI heavily accelerated the testing process.

After the hardware and software were ready, the next thing was to collect datasets and train the models. The system was used to benchmark three traditional machine learning classifiers against a 1D-CNN model to map morphological signals to specific mechanical actions defined by the used in GUI. By comparing these algorithms, the project seeks to prove whether a low-cost, muscle-deformation-controlled system can eventually drive a practical open-source bionic hand.

Physical morphology sensing offers a reliable alternative to overcoming the hardware limitations of electrophysiological sensors. When muscles contract, they don't just release electrical signals; the shape of the muscle belly also undergoes physical changes and expands. Morphological tracking methods, such as force electromyography (FMG) or Hall effect sensors, can directly measure these mechanical changes without reading voltages from the skin [23], [24].

This deformation-based device converts muscle movement into analogue data. Permanent magnets are attached to the skin above the target muscle. When the muscle contracts and bulges, it pushes the skin outward, causing the magnets to move to the same horizontal position. Hall effect sensors surrounding the outside continuously measure the change in distance between these magnets by reading the magnetic flux. This completely avoids the major problems of surface electromyography (sEMG). Since it tracks physical motion rather than microcurrents, the sensor does not detect skin impedance, sweat, or room humidity [25]. Furthermore, the frequency of muscle bulging is much lower than the firing frequency of motor unit action potentials (MUAPs). This means that low-cost hardware will not inadvertently filter out useful data, thus maintaining the integrity of physiological signals without the need for expensive analogue amplifiers.

2. Background

2.1. Evolution and classification of dexterous robotic hands

Dexterous hands are high-degree-of-freedom (DOF) robotic end-effectors that mimic the structure and function of the human hand. Through intricate biomimetic structures and actuation methods, they simulate the flexible movement and fine control of human hands, possessing the potential to perform human-like operations. The development of the dexterous robotic hand is to mimic human hands as closely as possible, including appearance, flexibility, adaptability, and complexity. Another reflection of flexibility is DOF, the number of independent movements that can be achieved. As established in the Introduction, the human hand possesses up to 27 DOF.

Robot grippers started with two fingers but commonly evolved to three; the Okada dexterous hand designed in 1979, the Stanford/JPL hand, and the Utah/MIT hand in the early 1980s were the representatives of dexterous hands. They have three fingers with DOF varying between 9-25. In the 1990s, NASA designed robotic dexterous hands for space operations: the Robonaut hand [26]. These dexterous hands have 5 fingers with 14 DOF. Nowadays, companies like Tesla or Yushu (Unitree) are providing innovative designs for dexterous hands with higher DOF (such as the Optimus hand with 22 DOF and Dex5 with 20 DOF), integrated sensing capabilities, tendon-driven architectures [27], and reverse drive [28].

The mechanical design of dexterous hands essentially involves striking a balance between adaptability, coupling and stiffness. In terms of adaptability, underactuated and fully actuated designs represent two distinct paradigms. Underactuated robotic arms rely on basic mechanical linkages. This structure enables the fingers to wrap around objects without the need for complex control commands. Underactuated robotic arms are simple in structure, resulting in relatively low manufacturing and maintenance costs [29]. Fully-driven robotic arms, on the other hand, adopt the opposite approach. Designers install dedicated motors at each major joint

to ensure the highest precision. However, this rigid control scheme eliminates passive adaptability, making it difficult for the robotic arm to grasp irregularly shaped objects naturally.

Table 1. Comparison between underactuated and fully actuated

Aspect	Underactuated	Fully actuated
Complexity and cost	Lower (fewer motors, simple control)	Higher (motors for major joints)
Control	Simple (command for grab/open only)	Complex (required precise coordination of all joints)
Adaptability	High passive adaptability	Low passive adaptability
Robustness	High (simpler mechanism, less to break)	Can be lower (more complex components)
Precision	Lower (cannot control every joint angle directly)	High (direct control over every joint)
Primary Goal	Robust, stable grasping of a wide variety of shapes	Precise, in-hand manipulation

The balance of stiffness is another key trade-off in mechanical design. A robotic arm requires a certain degree of structural stiffness to transmit power and remain stable under load. However, it also requires a certain degree of mechanical flexibility to absorb physical impacts and safely adapt to different object shapes.

The transmission mechanism decides the physical limits of the end-effector. This architectural choice directly dictates the hand's final weight, physical size, output force, and back-drivability [30]. Rigid transmissions, built with standard linkages or gears, excel at transmitting pure force.

They hold their position reliably, making them the standard choice for high-precision tasks.

Tendon-driven systems, on the other hand, use cables to transmit force, significantly reducing the robot hand's weight and inertia, allowing designs to approach human-like forms and agility.

The potential issues for this system are friction, cable stretching, a complicated cable channel design, and the need for continuous tensioning, which may affect control accuracy and long-term reliability.

Table 2. Comparison between tendon-driven and linkage/gear-driven

Aspect	Tendon-Driven	Linkage/Gear-Driven
Core Principle	Transmits force and motion remotely via cables (tendons), similar to biological tendons	Transmits force and motion directly through rigid components (gears, linkages)
Actuator Placement	Actuators can be remotely located, drastically reducing the weight of the end-effector	Actuators are typically mounted at or near the joint, increasing the weight and inertia of the moving parts
Precision & Stiffness	Lower, easy to elastic stretch, with friction, hysteresis, and lower overall stiffness	High, offers high rigidity, high transmission efficiency, low backlash, and excellent positional accuracy
Back drivability	Generally poor	Generally good
Force/Tactile Sensing	Joint torque can be estimated indirectly by measuring tendon tension, though friction can corrupt the signal	Typically requires force/torque sensors to be mounted directly on the joint, adding complexity
Inherent Compliance & Safety	Inherently compliant, elasticity of tendons can absorb shocks, making interactions safer	Inherently rigid, achieving compliance requires complex active force control algorithms
Complexity & Maintenance	Complexes require frequent maintenance	Structurally complex but highly reliable, requires less frequent maintenance
Typical Applications	Dexterous hands, lightweight robot arms, bio-inspired robots	Industrial robot arms, high-precision CNC machines, articulated robots

In summary, there is no single optimal design solution; the best choice entirely depends on the robot's intended tasks and application scenarios. For open-source foundational research and low-cost prototyping, an underactuated, tendon-driven architecture provides the necessary biomimetic compliance and space utilisation while remaining highly cost-effective. This project used the open-source TetherIA Aero Hand Open as the physical end-effector.

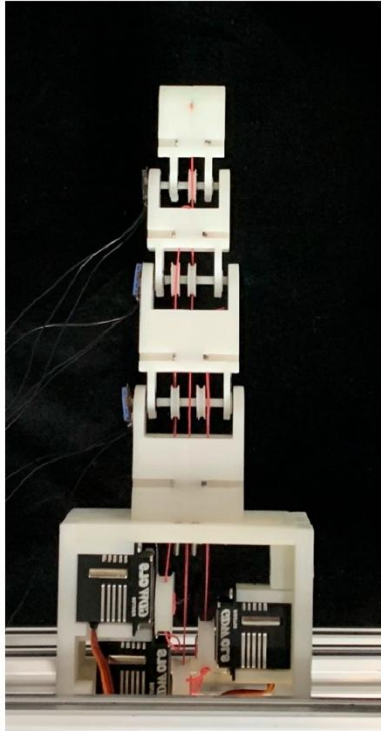


Figure 1. Demonstration of tendon-driven finger [31]

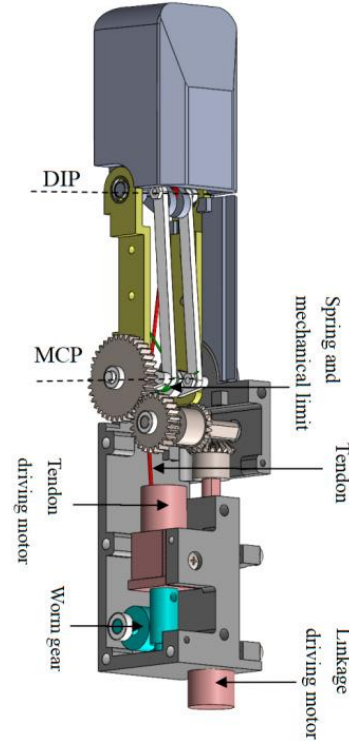


Figure 2. Demonstration of Linkage/gear-Driven finger [32]

2.2. Data collection units

First, we must understand how muscles generate signals. The central nervous system triggers voluntary movement. Motor neurons transmit action potentials to muscle fibres, forcing the fibres to depolarise and contract. When these individual motor unit action potentials (MUAPs) overlap and travel through the tissue to reach the skin surface, they form a complex and broadband electrical signal. [33].

For decades, surface electromyography (sEMG) has been used to capture electrical signals. Today, high-density surface electromyography (HD-sEMG) has become the industry benchmark [34]. Leading prosthetic developers, such as BrainCo and Ottobock, map the muscle belly using dense, two-dimensional electrode arrays. This massive spatial resolution allows digital signal processors (DSPs) to break down and separate individual MUAPs. This detailed mapping allows a robotic hand to control each finger independently and continuously [21].

But these solutions are all too expensive. Reading microvolt-level sEMG signals requires a highly specialised Analogue Front-End (AFE). The circuit needs an exceptional Common-Mode Rejection Ratio (CMRR) and precise broadband filters (usually 20 to 500 Hz). These filters must block ambient electromagnetic noise while preserving the non-linear, high-frequency spikes of the MUAPs [33]. As Table 3 shows, cheap bioelectric boards like the AD8232 are built for low-frequency heart monitoring. Their hardware filters actively delete the critical high-frequency muscle data. Once the hardware cuts this data out, no software algorithm can decode the user's intent.

Furthermore, electrophysiological signals are inherently physically unstable. Skin impedance fluctuates constantly in response to environmental conditions. When a user sweats, the sweat forms a mobile dielectric layer, thereby altering the skin's electrical conductivity and causing significant baseline drift [19]. Consequently, maintaining signal stability over extended periods of use remains a major engineering challenge, even for high-end commercial products.

Table 3. Technical requirements and bottlenecks of sEMG acquisition architectures

Architectural Metric	High-density sEMG (commercial benchmark)	Low-cost sEMG (e.g., AD8232/open source)
Spatial resolution	High (two-dimensional multi-channel arrays)	Low (single or dual-channel discrete configurations)
Hardware bandwidth	Broadband (typically 20-500 Hz)	Narrowband (Hardware-truncated, typically < 40Hz)
Signal integrity	Preserves complex MUAP high-frequency features	Loss of non-linear high-frequency discharge details
Electrode interface	High-fidelity Ag/AgCl or specialised conductive polymers)	Susceptible to impedance fluctuations (e.g., dry electrodes)
System cost complexity	Extremely high (requires dedicated DSPs and amplification)	Low (economically viable for foundational research)

Physical deformation sensing provides an alternative approach to overcome the strict hardware and environmental limitations of surface electromyography (sEMG). When motor neurons fire,

muscle fibres not only generate current, but the entire muscle belly also undergoes physical expansion and displacement. Methods such as force electromyography (FMG) and muscle deformation tracking can directly measure these physical morphological changes without measuring the voltage on the skin surface [23], [24].

When muscles contract, they push against the skin, causing adjacent magnets to move. A Hall effect sensor array measures muscle deformation by reading the changing magnetic flux. Because the sensors ignore the skin's microcurrents, factors such as indoor humidity, sweat, or changes in skin impedance do not interfere with the data. This method of measuring mechanical motion completely bypasses the problems encountered by sEMG [25]. This mechanical approach keeps the raw physiological signal intact without relying on high-end, expensive analogue circuits.

Neither raw data from a high-density surface electromyography (SEMG) array nor from a Hall effect armband can independently drive the robotic hand. The system requires a signal recognition algorithm to convert these continuous real-time signals into control commands for the robotic hand. Engineers use supervised machine learning classifiers like Support Vector Machines (SVM), Random Forests (RF), and other lightweight neural networks for human-machine interfaces [35]. During training, these models learn to map specific extracted features of the signal to target labels. Once trained, the model can be deployed to read the intention signals and translate them into bionic hand control commands in real time.

2.3. Ground truth of the dataset

During the training phase, the software must perfectly synchronise those signals with highly accurate mechanical target labels. For supervised machine learning, obtaining this absolute 'ground truth' of hand kinematics, such as exact finger-closure distances, is a hard prerequisite. If the model lacks accurate targets to learn from, it cannot classify anything.

Usually, engineers capture this high-fidelity data using expensive tracking gear. This usually

means relying on sensory data gloves equipped with bend sensors or setting up multi-camera optical motion capture rigs such as Vicon [22]. These premium systems easily provide high-frequency, continuous data logging. However, their high costs, fragile components, and stringent calibration requirements make them unsuitable for this project.

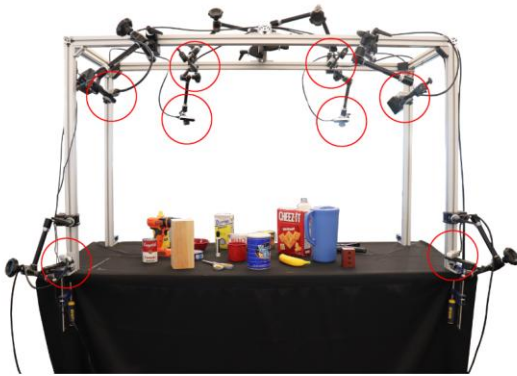


Figure 3. A typical multi-camera RGB-D capture rig designed for high-fidelity, markerless hand kinematic tracking [38]



Figure 4. High-fidelity commercial motion capture gloves (Manus Quantum Metagloves) representing a high financial barrier for open-source bionic research [39]

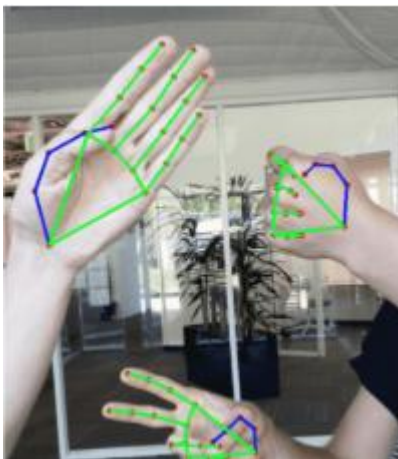


Figure 5. Real-time multi-hand tracking [44]

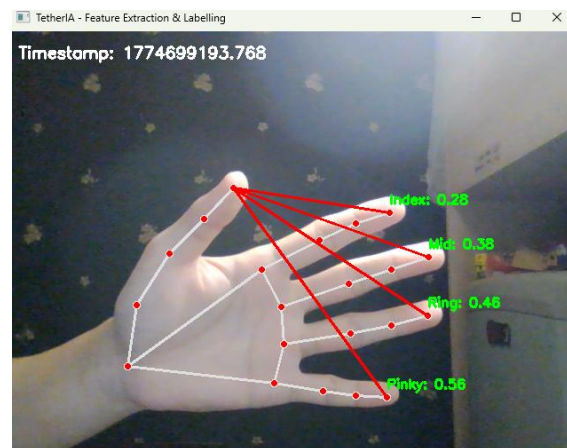


Figure 6. Real-time hand tracking with finger distances and timestamp

To get around these high costs, researchers often use computer vision tracking as an automated labelling tool. Deep learning pipelines like Google's MediaPipe provide a cheap and accessible alternative. They use standard RGB webcams to pull 3D hand landmarks in real-time without

needing any physical markers glued to the user [40], [41]. When building bio-signal datasets, these vision frameworks theoretically act as automated supervisors. They watch the physical hand movements and map them directly to the incoming physiological data. [42]. Tracking the user's hand during data collection allows the software to automatically assign discrete state labels or continuous trajectories. This theoretically cuts down on hours of manual data tagging.

3. Methods

3.1. Bionic hand assembly

To mimic the structure of a human hand as closely as possible, the robotic arm must conform to standard human proportions and structural geometry. Commercially available dexterous robotic arms are prohibitively expensive, completely contradicting the project's stringent low-cost requirements. Therefore, we focused entirely on open-source hardware platforms.

Open-source designs are typically categorised into two types: tendon-driven systems and rigid link or gear-driven systems. Gear-driven mechanisms offer excellent kinematic precision and rigid force transmission. However, achieving this mechanical rigidity requires bulky structural components, resulting in an overall size that far exceeds that of a normal human hand. Tendon-driven mechanisms address this issue by relocating the bulky motors away from the finger joints. They utilise tensioned cables to pull the joints, mimicking the functioning of biological muscle fibres and tendons. Although budget-friendly systems cannot perfectly replicate the complexity of human hand kinematics, this tendon-driven approach allows the fingers to remain extremely compact and anatomically correct.

After weighing up physical realism against strict budgetary constraints, the project ultimately selected the open-source TetherIA Aero Hand Open as the end-effector. Its codebase provides a fully 3D-printable framework designed specifically for robotic hand research. The Aero Hand Open employs a tendon-guided system driven by seven standard servo motors. By integrating these relatively heavy actuators into the palm, the design prevents mechanical mass from affecting the distal joints. This low-inertia design successfully drives the hand's seven degrees of freedom (DOF).

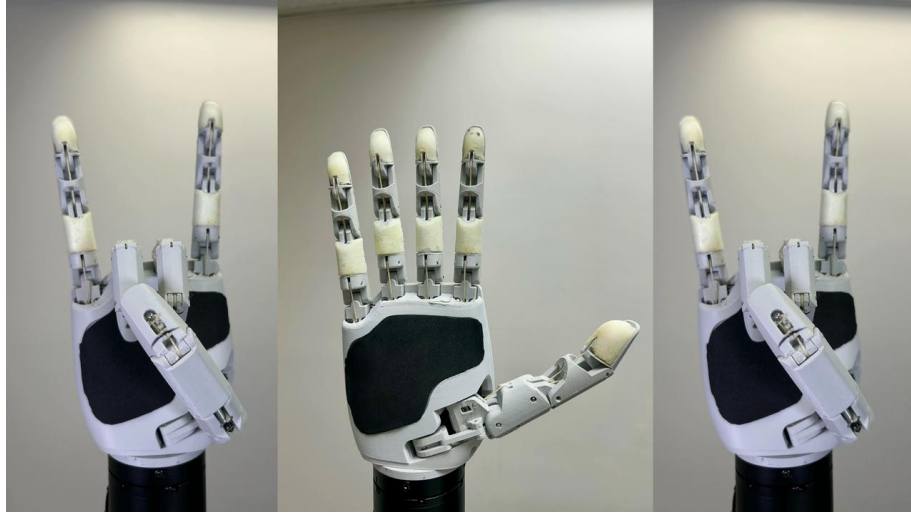


Figure 7. Overview of hand [45]

Subsequent checks revealed that the provided bill of materials (BOM) contained several missing or mismatched components, which may be due to the fact that this open-source project is still in the development phase. Furthermore, the original BOM relied heavily on the North American supply chain and required custom-manufactured hardware. To address these issues, we have created a revised bill of materials (detailed in Appendix A Table 6). The revisions included identifying missing components, removing redundant items and replacing bespoke parts with standard commercial alternatives. This process resolved supply issues whilst ensuring the integrity of the mechanical and kinematic designs.

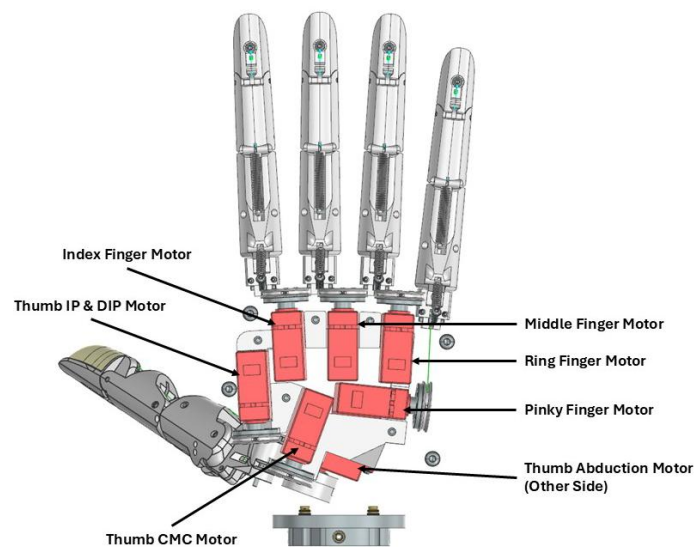


Figure 8. Overview of palm [45]

Meanwhile, the 3D-printed components have also been completed. The palm and fingers were 3D-printed directly from the provided .3mf files, without the need for any modifications to the CAD model. Printing parameters, such as infill density and orientation, were identical to the official specifications. The remaining hardware was sourced via regional e-commerce platforms such as Taobao and JD.com. Across all procurement and manufacturing stages, the servo motors were the most expensive component and represented the sole major expenditure for the entire project. Within the domestic Chinese supply chain, the cost of components such as screws, bearings and pull wires was virtually negligible. This low-cost procurement process demonstrates the viability of the TetherIA platform as a low-cost research infrastructure.

Once all the 3D-printed parts and purchased hardware were ready, the physical assembly began. Assembling this hand required two tasks to be carried out simultaneously: assembling the mechanical structure and soldering the electronic components. As the manufacture of custom PCBs was beyond my capabilities, this task was undertaken by the company's technical staff. As for the mechanical assembly, an inspection of the 3D-printed parts revealed that some had significant dimensional tolerances, rough surfaces and burrs. These defects were likely caused by a decline in the 3D printer's precision during prolonged printing. To avoid wasting time later on, I also printed a second complete right hand as a spare. As the four fingers in the TetherIA design share the same geometry, the spare parts are highly interchangeable, making this task straightforward to complete.

The assembly of the bionic hand begins with manually sanding the 3D-printed joints to smooth the surfaces and reduce mechanical friction. This is followed by the routing of the tendons. Each finger consists of five separate sections and requires a dual-tendon system. One active tendon pulls the finger inwards to close it via a motor, whilst the other passive elastic tendon is fixed inside the finger and pulls it back to the open position. Although TetherIA provides an installation manual, the details are presented solely as CAD side views, which can easily lead

to wiring errors; if handled incorrectly, this may require the complete reconstruction of several fingers.

The tight dimensional tolerances between the micro bearings, metal pivot pins and the 3D-printed plastic housing have also led to serious alignment issues. Some bearings became completely jammed during installation. The printed structure had to be destroyed in order to remove the jammed bearings, which is why it is necessary to print extra finger structures as spares from the outset.



Figure 9. Components and printing parts initially

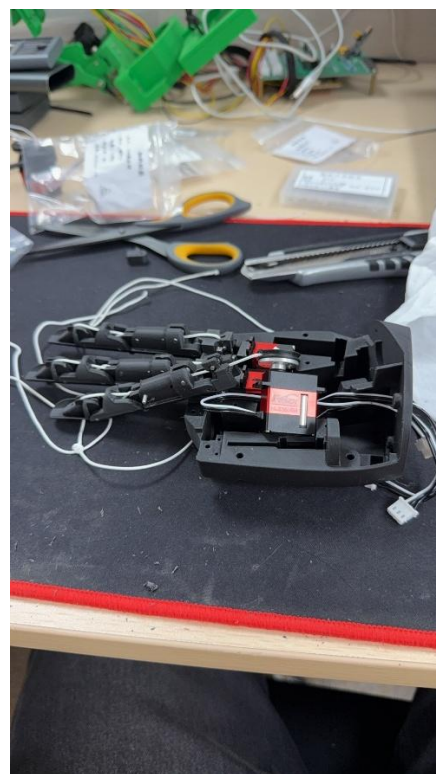


Figure 10. Assembly over servo motors

With the assembly of the bionic hand largely complete, the finished PCB and seven servo motors needed to be fitted into the bionic hand. As the bionic hand is similar in size to a human palm, the internal space is extremely confined. The seven motors take up almost all the space, leaving virtually no room for the control cables. Such a compact layout exposes a critical design flaw: the palm-shaped housing lacks any active or passive cooling mechanism. Early hardware testing indicated that these servo motors generate significant heat during operation. Enclosing seven running motors within a 3D-printed plastic housing poses a substantial risk of

overheating and actuator burnout. To properly manage these hardware risks, I compiled a Failure Mode, Effects and Criticality Analysis (FMECA) report (detailed in Appendix A Table 7 and Table 8). This report lists a number of issues identified during the assembly process, to serve as a reference for future testing.

3.2. Signal acquisition system development

The first-generation acquisition unit utilised an AD8232 analogue front-end module connected to an ESP32 microcontroller for testing sEMG. The initial test code was written and uploaded in the Arduino IDE. Subsequent testing confirmed that the breadboard connections were functioning correctly; the ESP32 successfully read the raw analogue input signals and transmitted them in real time to the Arduino serial plotter. Table 4 lists the specific pin configuration used for this initial circuit check.

Table 4. Pin mapping for initial AD8232 sEMG testing configuration

AD8232 Module Pin	ESP-32D Microcontroller Pin	Signal Type/Function
3.3V	3.3V	Power Supply (3.3V DC)
GND	GND	Common Ground
OUTPUT	D 34	Analogue signal output

Analysis of the collected data revealed that some data features were missing, and the problem was later traced to the AD8232 chips. Because this board is built specifically for cardiac ECG tracking, it features an aggressive hardware low-pass filter. The filtering automatically removes high-frequency signals, which is undoubtedly a disaster for gesture recognition. Gesture classification learning relies on the signal characteristics of force exerted during a gesture, including peak values. Due to this filtering, a large portion of the features of some gestures may be filtered out, making them difficult to learn or confusing. While this chip might be used as a recognition threshold, such as for a natural hand position and a clenched fist, recognising only two gestures is insufficient for controlling a bionic hand.

Apart from the frequency truncation, the raw data streams also suffered from severe electrical noise and erratic voltage fluctuations. The ECG module relied on disposable wet electrode

patches that quickly lost adhesion, stripping off and requiring a fresh set of patches before each test run, which was very inconvenient for system operation.

To overcome the limitations of hardware filtering and avoid wasting money on disposable patches, the system was upgraded to a dual-channel dry electrode surface electromyography (sEMG) module. The purpose of switching to dry electrode sensors was to extend the recording frequency band without affecting the onboard circuitry, while completely eliminating the need for consumable gel patches.

By directly connecting the dual-channel dry electrode sEMG board to the ESP32, this gel-free solution could be tested immediately. As shown in Table 5, the new module simplifies electrical integration to just four physical connection lines.

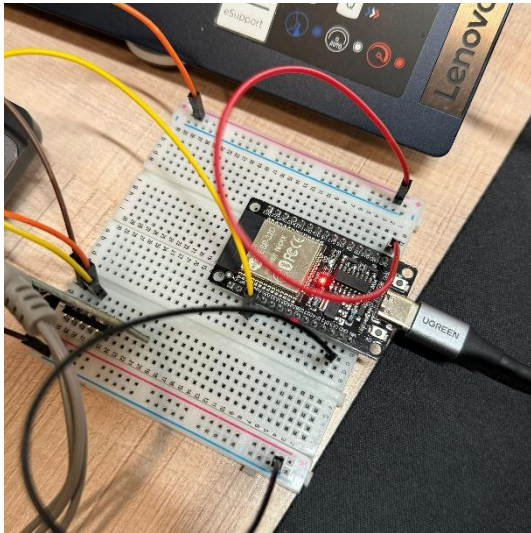


Figure 11. Breadboard setup for AD8232 unit

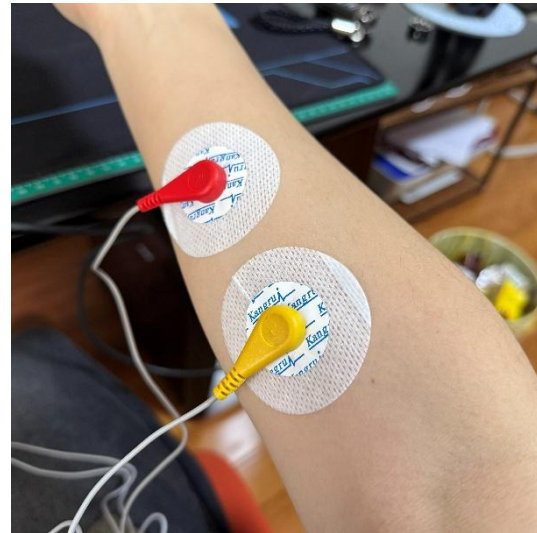


Figure 12. Demonstration of wet electrode patches on forearm

Table 5. Pin mapping for dual-channel dry-electrode sEMG configuration

Dry Electrode module pin	ESP-32D Microcontroller Pin	Signal Type/Function
VIN	3.3V	Power supply
GND	GND	Common Ground
CH1 OUT	D 35	Analogue voltage output for channel 1
CH2 OUT	D 34	Analogue voltage output for channel 2

For this second iteration, firmware development moved away from the basic Arduino IDE. The

project switched to visual studio code (VS Code) with the PlatformIO extension installed. A quick custom script was written to open a basic PC window, allowing the system to plot the raw signal fluctuations in real time. (The full development of the data logging GUI will be detailed in the section below)

During physical bench testing, each dry-electrode sensor had to be strapped to the forearm with its own Velcro band. Extended use quickly exposed a chain of physical problems. Securing two separate bands meant constantly squeezing the forearm. However, several problems were discovered after a long time of wear. Because two separate straps were tied parallel to the arm for extended periods of data collection, muscles easily became sore and fatigued. This caused fluctuations, instability, and misalignment in muscle signals. Loosening the straps, on the other hand, prevented the dry electrodes from maintaining a tight fit against the skin, potentially introducing environmental noise and further contaminating the collected data.

As the tests were conducted longer, sweat and physical movement also destroyed the signal baseline. Dry electrodes are incredibly sensitive to those situations. Even a slight physical shift caused massive voltage spikes that completely contaminate the recorded dataset. Apart from the electrode movement issue, the DIY breadboard was another problem. Although the breadboard offers enough operational space and easy adjustment, the wiring connections are not entirely secure, as signal acquisition and the chip are still connected via data cables. This inevitably leads to pulling on the data cables during data acquisition, causing the breadboard wiring to loosen. This undoubtedly increases noise in the received signal, further contaminating the dataset.

Early machine learning tests also started during this hardware phase. New problems appeared when testing the collected dataset. The information captured by the dual channels was very limited, and for some complex gestures, such as pinching, the required muscle information might not be sufficient. Without capturing this relevant information, the model could not learn

and analyse properly. Therefore, to obtain more information, the number of channels needed to be increased to collect information from more muscle regions of the forearm.

Commercial multichannel surface electromyography (sEMG) equipment is mostly very expensive and closed-source. After researching and analysing another type of signal acquisition method, muscle deformation signals, a search was conducted for related products. It was found that, due to differences in principle, its cost was relatively low, and products that allowed for secondary development could also be purchased. Therefore, after investigation and analysis, it was decided to replace the third-iteration signal acquisition unit to track muscle deformation.

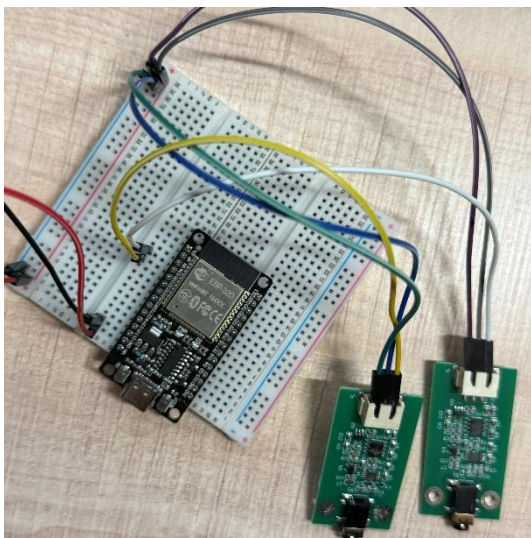


Figure 13. Breadboard setup for sEMG sensors

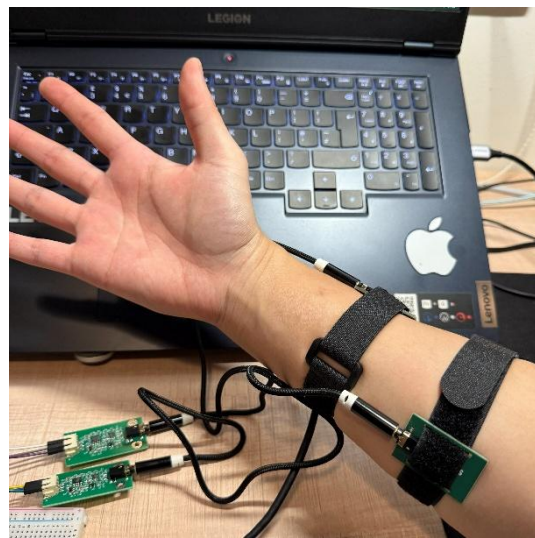


Figure 14. Overview of dual-channel sensors over forearm

The final hardware setup relies on the Feimazuo muscle deformation armband. This device tracks movement using an array of Hall-effect sensors and permanent magnets. When the forearm muscles contract, the muscle bellies bulge outward. The armband detects changes in physical distance by reading shifts in the magnetic field. Because it measures mechanical movement rather than microvoltages, the setup delivers a high signal-to-noise ratio compared to the sEMG system. This purely physical approach completely ignores the electrical biometric noise that plagued the previous hardware iterations.

The device has six independent data channels. Due to purchase restrictions, disassembly and

modification of the device are not permitted. Instead, it serves purely as an external data-collection tool to feed the custom software. Before pushing any data to the computer, the onboard circuit runs a preset filtering script to clean up the raw analogue-to-digital converter (ADC) values.

Unlike earlier breadboard and USB setups, the muscle deformation armband transmits data wirelessly. A dedicated Bluetooth adapter transmits the six-channel readings to the PC at a steady 31 Hz. Removing the physical USB cable eliminates ambient electrical noise and makes the hardware layout cleaner.

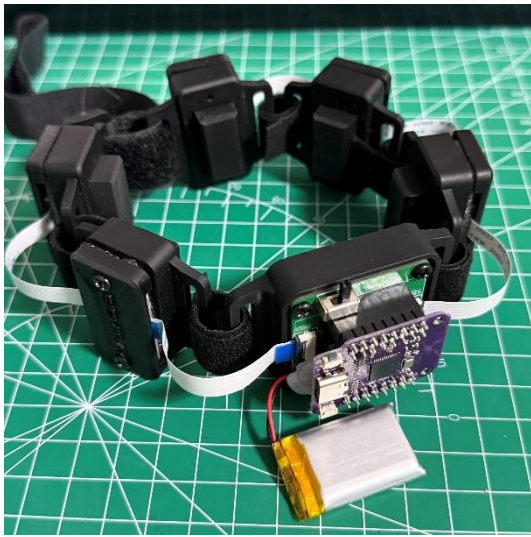


Figure 15. Overview of muscle deformation armband



Figure 16. Demonstration of armband over forearm

3.3. Software architecture and GUI

Early hardware testing relied on several independent scripts in VS Code. For example, the real-time waveform plotting window and the visual capture window required separate scripts to run in the terminal to work together. This resulted in numerous operations for each data collection session, leading to inefficiency. Therefore, to streamline data collection, a graphical user interface (GUI) with multiple functional modules was developed to centralise all these functions in a single window for unified operation.

Regarding data acquisition and labelling, since the goal is to classify discrete gestures, precise labelling of continuous data is unnecessary. This means that highly accurate and expensive

equipment is not required to capture gesture movements, such as tracking the spatial movement of knuckles. Therefore, a standard webcam, such as a laptop webcam, and a vision system based on the MediaPipe framework are sufficient. This vision system automatically identifies the user's hand (in this case, the right hand) and captures 21 hand feature points. It then uses the distance between each finger and the tip of the thumb to recognise different gestures. This visual capture system is synchronised with the signal acquisition system. Once the data acquisition function is activated, the visual capture system will label the data with real-world data according to regulations.

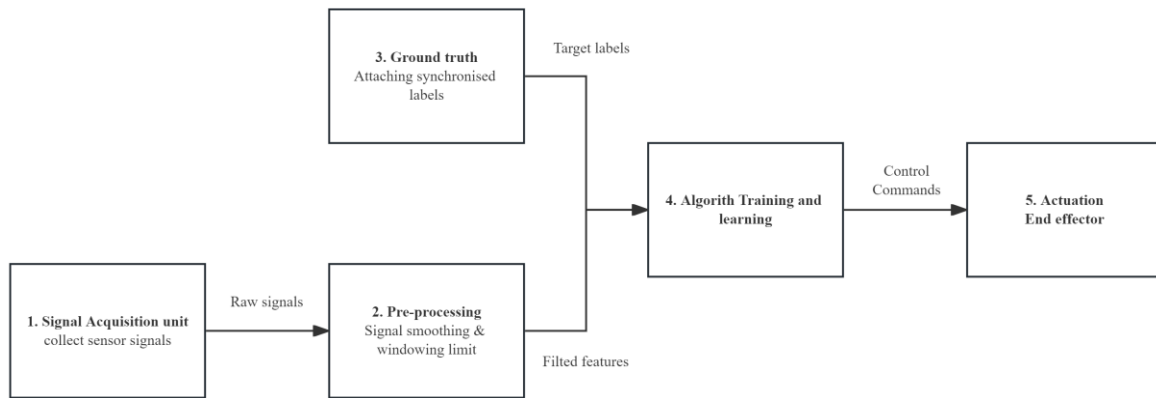


Figure 17. State machine logic and data logging workflow

As Figure 18 shows, the early GUI layout was split into different functional modules. The left side of the window plotted the real-time filtered signal waveforms. The right side displayed the live webcam feed with the 21 MediaPipe landmarks overlaid, along with two text prompts in the top-right corner. The blue text indicated which target gesture to perform if collection is started. The green text indicated the gesture that the visual model is currently detecting. The visual capture system only initiates data recording when the detected visual state perfectly matches the target.

The right-hand area contains multiple drop-down menus for selecting algorithms and mapping different gestures to specific actions. Below are calibration and start buttons. In this initial phase, the acquisition protocol only captures four different gestures, with each gesture captured

20 times. To accurately capture gesture signals, the acquisition system records the signals within one second before and after the correct gesture is detected and labels them accordingly. Because standard machine learning models struggle to process raw, high-dimensional time-series signals directly, the recorded data required feature engineering. The continuous data streams were sliced into fixed-length windows. During the early tests with the dual-channel dry-electrode sEMG, the system mathematically extracted five time-domain metrics from each channel: root mean square, mean absolute value, waveform length, zero-crossing rate, and standard deviation.

The original dataset collection protocol imported the entire recording session, which contained multiple continuous gestures, into a single csv data file. After a long time of collection, this continuous tracking method quickly failed. Forcing the user to perform dozens of back-to-back repetitions without any breaks caused severe muscle fatigue. As explained in Section 2, this will result in a complete drift of the signal baseline. Furthermore, repeating the same gesture without resetting the hand posture created overly uniform data. The resulting dataset lacked the statistical variance needed to train a robust machine learning classifier.

To avoid these problems encountered during collection, a new data collection protocol has been drafted. To avoid the previous long periods of continuous data collection, the new protocol adopts a discrete, key-triggered, single-collection mode. The collected data is no longer stored in a single CSV file, but rather in separate folders created for different gestures. The data from each collection is stored as plain text (.txt) in the corresponding gesture's folder.

The discrete data acquisition method primarily addresses two issues. One is the muscle fatigue caused by prolonged continuous collection, as mentioned earlier. The other is that this scheme completely bypasses the visual capture system. While the accuracy requirements for visual capture are not very high, in practice, it was found that there are still certain requirements regarding the camera angle and lighting. This is because the visual capture system captures and

calculates 21 points on the user's hand. However, some actions or lighting conditions can prevent the system from recognising gestures, such as pinching. Fingers may be obscured due to these reasons, causing the system to fail to recognise the trigger gesture, preventing data collection from starting according to the old protocol, or resulting in incorrect identification and labelling, thus polluting the dataset. The new protocol adds new functional modules. Users can first select the gesture to be collected, then adjust their arm, and actively trigger the spacebar to start a single data collection session. This ensures accurate labelling because the decision-making and control are in the user's hands. Furthermore, this allows users sufficient rest time during data collection, preventing muscle fatigue from affecting the signal. To improve data diversity and control variables, a collection protocol (detailed in Appendix B) must be followed. This protocol specifies active adjustments during collection, such as force and angle, to inject necessary statistical variance into the dataset.

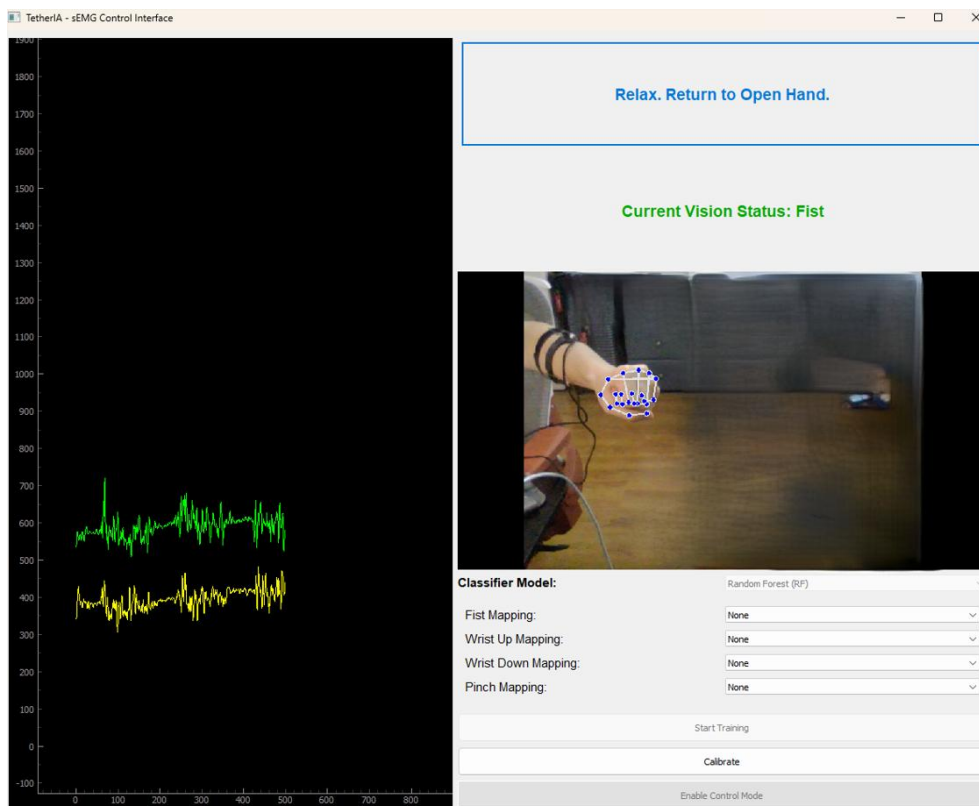


Figure 18. Early version of GUI

To handle these new protocols, the GUI was completely rebuilt. Figure 19 shows the final

modular layout. The left side remains signal visualisation, displaying six independent, grid-enabled plots for the six channels of the muscle deformation armband. The right side contains the operational controls. The top-right corner still shows the camera and MediaPipe overlay, but it now serves only as an offline observational tool. It is completely disconnected from the actual data-logging trigger or tagging ground truth. Just below the camera is the Bluetooth Serial connection panel, which lets the user manually set the COM port and baud rate.

Next is the main data recording module. The user selects one of five target gestures from a dropdown menu. Below the dropdown menu is a red text prompt; when the user presses the spacebar to trigger the data collection function, the text turns green, indicating that data collection is in progress and that it accurately records data within 2.5 seconds of initiation. Once collection is complete, the text turns red again, and the GUI automatically saves the collected data to the corresponding gesture's folder. (For details of the underlying multithreaded code that handles this asynchronous data stream and renders the UI, please refer to Appendix C Algorithm 1).

The layout also includes a reserved robotic mapping panel to link predicted gestures to mechanical motor commands. Right now, this acts purely as a UI placeholder for future physical actuation.

A 'Live Feedback' box is located at the bottom right corner to track real-time performance. It compares what the MediaPipe vision system sees against what the loaded machine learning model predicts, displaying the model's live confidence percentage. Again, the MediaPipe system only provides results, not tagging the ground-truth tags to the datasets. The two buttons above the live feedback box are the Evaluate Dataset File button and a live system bottom. Early versions of the GUI included a 'baseline calibration' button to try and fix signal drift. However, real-time testing proved this function was completely useless against physical sensor drift, so it was deleted.

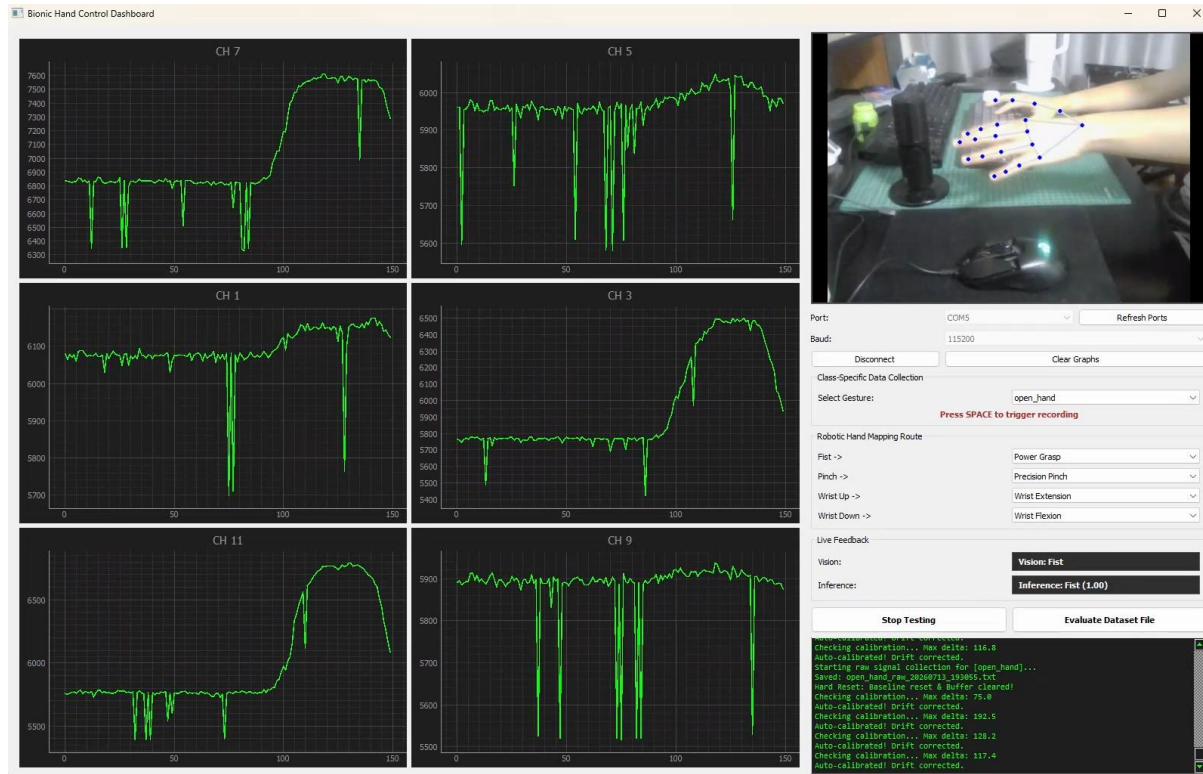


Figure 19. Final version of GUI

The final version's GUI emphasises model evaluation. Therefore, it includes an "Evaluate Dataset File" button for testing offline data. Its function is to allow users to perform a simulated real-time sliding-window test on the offline dataset after selecting and loading a trained model by clicking "live system," even if the armband is charging or unavailable. This allows them to check the model's accuracy and output the results to the live feedback box. This is a useful feature if the user has a large dataset and a newly trained model.

3.4. Model architectures and offline data pipeline

The graphical user interface (GUI) records raw sensor data as a continuous data stream. To segment this continuous data stream into usable training data chunks, the offline script uses a sliding window to slice the data. Window parameters are strictly set:

- Window Length(N): 40 discrete sampling points
- Step size(S): 10 sampling points
- Overlap ratio: 75%. Sharing 30 data points across consecutive windows ensures the

system doesn't miss rapid, sudden hand twitches, while artificially increasing the size of the dataset used for model training [43].

After slicing the data, Figure 20 shows how the pipeline splits into two distinct processing tracks.

Track 1 is for 1D-CNN. This branch formats the raw time-series tensors into the format required by the custom deep learning network. First, the dataset is balanced using a random undersampler so that all five gesture categories have the same number of samples. Next, the script applies dynamic zero-mean normalisation independently to each window using the following method:

$$\text{Normalised Window} = \text{Window} - \text{np.mean}(\text{window}) \quad (1)$$

This simple mathematical trick eliminates the absolute baseline drift caused by armband movement, leaving only the relative waveform shape. These cleaned tensors can be directly fed into a lightweight one-dimensional convolutional neural network (1D-CNN). Complex deep neural networks consume significant computational resources and introduce latency. To ensure real-time control, this CNN employs a streamlined architecture. It stacks only two one-dimensional convolutional blocks to automatically extract spatiotemporal features, followed by a dropout layer (set to 0.3) to prevent the model from memorising training data. A final fully connected dense layer maps the extracted features to five target gestures. The network is trained using the standard Adam optimiser and cross-entropy loss function [46].

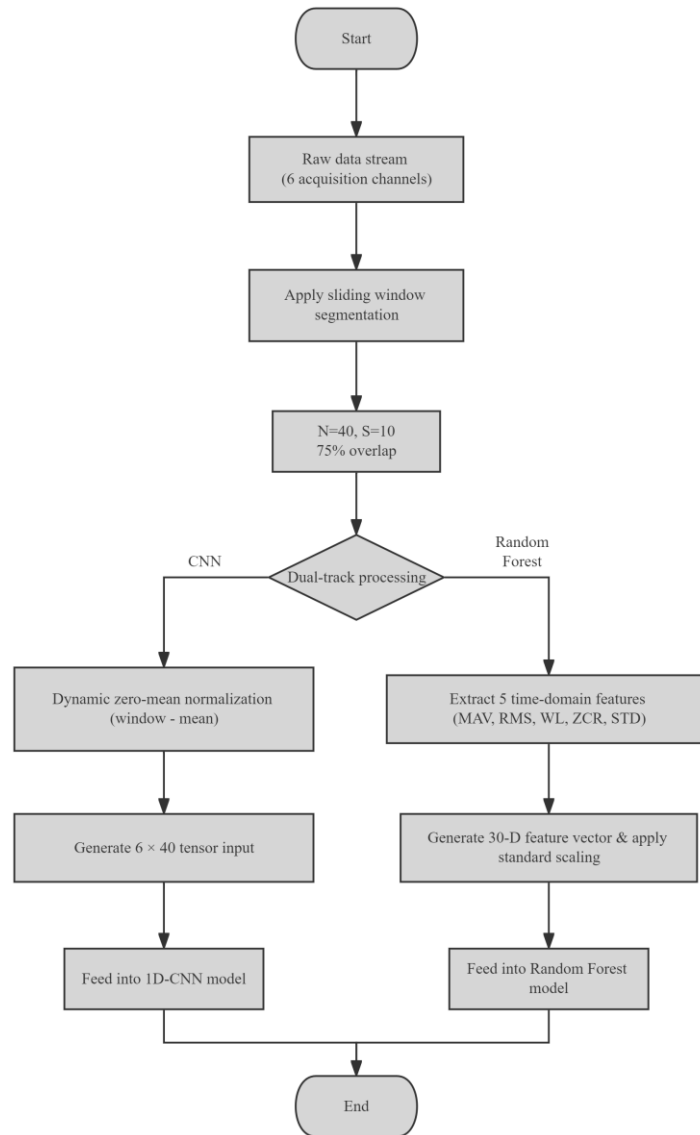


Figure 20. Flowchart of the offline preprocessing pipeline and classifier routing

Track 2 is for the Random Forest. Basic machine learning classifiers cannot easily chew through raw, multi-channel time-series arrays. To compress the data, this track mathematically extracts five time-domain features from each of the six sensor channels. This step spits out a neat 30-dimensional feature vector (6 channels $\times$ 5 features) for every single window[35].

Let x_i represents the i -th amplitude reading inside a single-channel window of length N , the five metrics are calculated as follows:

$$Mean = \bar{x} = \frac{1}{N} \sum_{i=1}^N x_i \quad (2)$$

$$STD = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2} \quad (3)$$

$$RMS = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad (4)$$

$$ZCR = \sum_{i=1}^{N-1} \mathbb{I}((x_i - \bar{x})(x_{i+1} - \bar{x}) < 0) \quad (5)$$

(where $\mathbb{I}$ is an indicator function)

$$WL = \sum_{i=1}^{N-1} |x_{i+1} - x_i| \quad (6)$$

Using the scikit-learn library, a baseline Random Forest classifier trains on these 30-dimensional vectors. The model uses an ensemble of 100 decision trees ($n_estimators = 100$) [47]. Before the data hits the trees, a standard scaler forces all features to have a zero mean and unit variance. Without this scaling step, metrics that naturally produce large numbers, e.g., Waveform Length, would completely dominate the decision-tree splits.

To benchmark this baseline model, the code runs a 5-fold stratified cross-validation ($k = 5$). This keeps the class ratios perfectly balanced across all folds. However, randomly splitting overlapping sliding windows creates a massive data leakage risk. The section below of this report heavily breaks down and exposes this exact overfitting problem [48].

4. Results and Discussion

4.1. Physical hardware evaluation and failure mode validation

Overall, the open-source TetherIA bionic hand is a good choice. Its standardised design makes it easy to manufacture and maintain. Structurally, it ensures necessary structural strength while maintaining a lightweight design. It is indeed more realistic than traditional robotic hands and has seven degrees of freedom. However, this open-source project also has some issues. Its entire structure is manufactured primarily by 3D printing, including the channels reserved for the cables. However, the channel structure is more complex than the entire dexterity hand, including bearings designed for low friction at corners. These features still cannot prevent friction when the rope is pulled, affecting both the 3D-printed parts and the bearings. Due to tolerance issues, most bearings do not fulfil their intended function of achieving low friction through rotation when the rope is stretched. In reality, the rope simply slides across its smooth surface without providing adequate low resistance. This leads to discontinuous motor operation and inability to reset.

Another issue with 3D printing concerns the printing material itself. The dexterity hand's design includes the need to drill screws at specific locations for fixation, but this screwing operation inevitably affects the overall structure, potentially causing structural cracking.

While tolerance issues are generally acceptable, observations suggest that some structural aspects require further examination. For example, the design of one degree of freedom in the thumb. This degree of freedom is achieved by directly linking the thumb to a servo motor, with the connection relying on a relatively thin link to allow the thumb to move from the side of the palm to the inside of the palm. During assembly, it was discovered that the component in this area experienced deformation due to tolerance issues. Upon completion of assembly and testing, it was found that when the servo motor was activated, the connecting rod had to withstand severe bending and shear stresses, rather than a clean, tensile stress. This undoubtedly increased

the risk of failure of the connecting rod structure and the servo motor.



Figure 21. Direct-drive rigid linkage of the thumb attached to the FeeTech servo, demonstrating a structural vulnerability to bending stresses

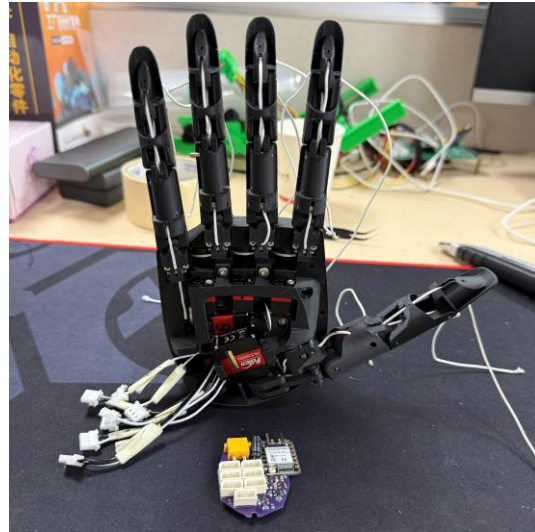


Figure 22. High-density integration of seven continuous-duty servo motors within the enclosed palm cavity, highlighting the spatial constraints that lead to thermal accumulation

However, the most serious problem has been identified as the servo motor's heat dissipation. Because the dexterous hand is designed to closely resemble the size of a real human hand, all seven servo motors are located within the palm. In addition to the servo motors, the palm contains structures such as finger connecting rods, which further reduce the space available for servo motor placement. Although the design included provisions for motor wiring, actual installation proved quite difficult, requiring the efficient use of every inch of space. Furthermore, the wiring for some servos was extremely complex, hindering future maintenance. Another significant issue was heat dissipation. The dexterous hand lacked any active or passive cooling system. As mentioned earlier, the thumb-based servo inevitably experienced rapid temperature increases due to significant resistance. While these servos were designed to maintain operation within a certain temperature range, prolonged operation without adequate cooling would inevitably lead to servo failure. Therefore, future designs may require a redesign of the dexterous hand's head section, optimising the layout and incorporating a cooling system, either active or passive, depending on the specific operating conditions and environment. For

example, adding a wrist section could relocate some motor PCBs and other electronic components to the wrist, optimising both space and heat dissipation. Further analysis is documented in the FMECA report. (Appendix A, Table 7).

4.2. Comparative analysis of signal acquisition hardware iterations

Early DIY designs for signal acquisition units focused on acquiring electrical signals from the skin surface, combining the AD8232 ECG and dry electrode sEMG modules. However, the inherent characteristics of the breadboard and connecting cables resulted in extremely poor overall resistance to environmental interference and instability. Despite improvements made to reinforce the cable connections and prevent breakage or loosening of contacts, it's still difficult to avoid cable pulling during actual data acquisition, which can introduce noise into the dataset. In the long run, this wired breadboard solution is unsuitable for long-term data acquisition.

For the muscle deformation armband, this design eliminates numerous cables and the breadboard-based design. It relies on a custom PCB board and flexible, anti-interference cables to connect the individual sensors. Furthermore, it supports Bluetooth data transmission, replacing the original USB data cable transmission. This allows the armband greater freedom of movement and operation during data acquisition, while the six channels provide a larger dataset, which is more beneficial for learning and training models like CNN.

However, this product also has shortcomings. The first is the structural design. Although this product is an armband, its six channels are connected in pairs via Velcro, ultimately wrapping around the arm. However, the channels themselves are quite large, and the overall structure and the sensor parts that contact the skin lack any ergonomic design. For example, the contact surface is a large rectangle instead of a curve designed to conform to the arm's surface. Even if the armband is successfully worn, it is not very comfortable, and the imperfect fit to the skin surface will affect the quality of the collected data to some extent. Under prolonged wear, the Velcro will naturally loosen due to continuous movement. This loosening will cause the sensor

to shift from its original contact position, which will undoubtedly affect the signal baseline and thus contaminate the data.

These are relatively minor potential impacts. During data collection, it was discovered that prolonged operation of the device causes severe heat generation in its chips, including its Bluetooth transmitter and receiver located near the computer. The visual impact is evident in the dataset. As plotted in Figure 23, after prolonged collection, all six channels exhibited highly unnatural signal spikes. Although previous electrode modules also generated heat, such pronounced and continuous fluctuations in the data were unprecedented.

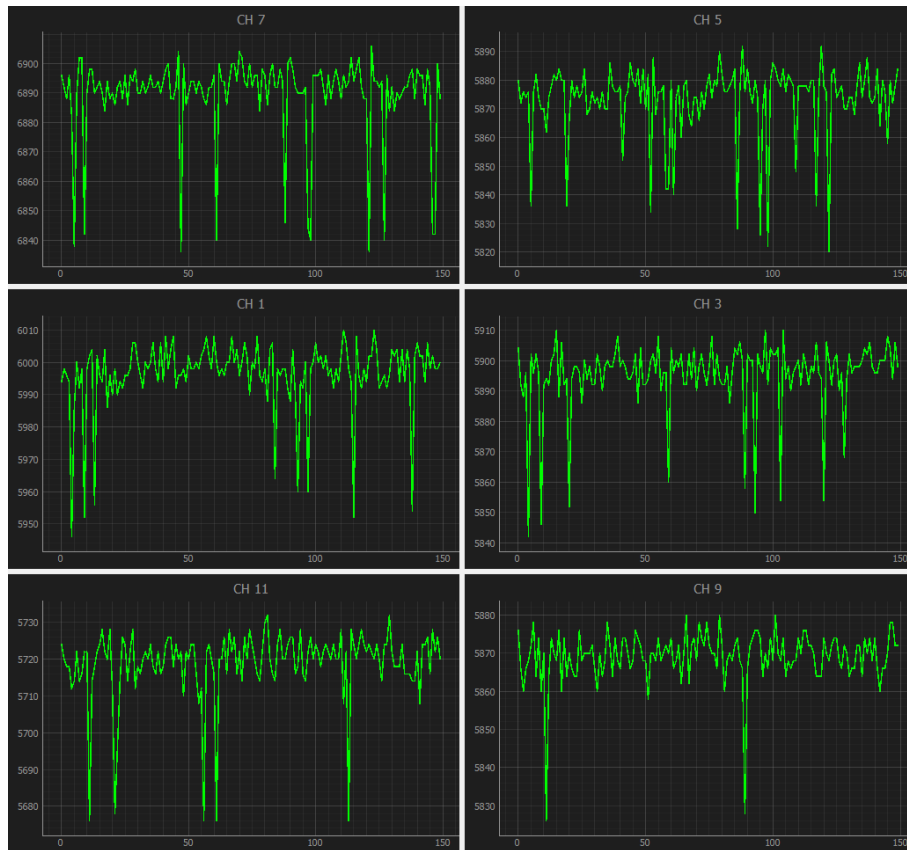


Figure 23. Unnatural signal spikes across all six channels of the muscle deformation armband, caused by thermal accumulation in the Bluetooth and sensor chips during extended operation

4.3. Feature space distribution and PCA analysis

When the collected dataset was used for offline training of individual models, most achieved extremely high accuracy rates. However, when a combined dataset with data from different time periods and potentially changed wearing positions was used for retraining, the previously

high accuracy rates plummeted. Real-time testing revealed that machine learning models often. To understand why this happened, principal component analysis (PCA) was performed on the dataset. Since five features were calculated for the signal to enable machine learning to better learn the signal features, PCA reduces the dimensionality of the high-dimensional feature array and projects the maximum statistical variance onto two orthogonal axes (PC1 and PC2) [35]. This approach facilitates the visual inspection of gesture classification results. Based on an analysis of a single data set, the features are relatively distinct.

Data points for individual gestures are typically closely clustered together, forming distinct clusters. Although some gestures overlap in their clusters due to similar force characteristics, they can generally still be distinguished. However, as shown in Figure 24, the situation becomes much more complex when data sets collected over different time periods are analysed collectively. The same gesture signals exhibit significant data drift across different datasets, resulting in distinct clusters. This clearly explains why data drift has such a severe negative impact on models, leading to a sharp decline in accuracy and making it difficult for real-time predictions to recognise individual gestures.

These significant statistical differences are attributed to temporal signal drift and mechanical sensor displacement [20]. As mentioned above, when these problems occur with the device, they will affect the dataset. For example, if the baseline value collected in the morning is collected again in the afternoon, the baseline value may have drifted by hundreds of primitive units, causing the training boundary to fail and making real-time gesture prediction impossible. To further eliminate this bias, the data acquisition process needed to be reconfigured. The first step was to abandon the function of assigning ground truth labels based on camera-based visual capture and recognition. Five specific hand gestures were then identified as data collection targets: fist, pinch, open hand, wrist up, and wrist down. Continuous data collection was replaced with time-discrete data collection. Furthermore, to ensure data stability while

improving data diversity, a standard data collection protocol was followed to ensure consistent data collection across time periods, thereby controlling for variables.

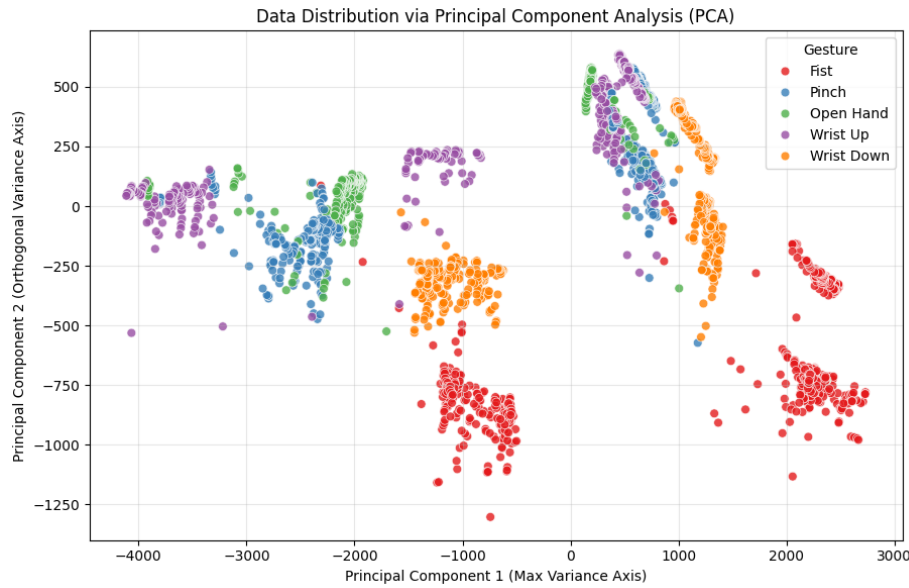


Figure 24. PCA scatter plots from multiple early datasets. Identical limb movements formed completely isolated subclusters in different datasets, revealing severe signal temporal drift

Following the implementation of the new acquisition protocol, a new PCA analysis was performed on the collected data, as shown in Figure 25. The increased diversity of the dataset due to the improved action diversity during acquisition also resulted in a significant improvement in dataset diversity. Box plots were then created for the 30 time-domain features of the dataset to verify their statistical stability. The Figure 26 shows that the standardised median was almost entirely distributed on the zero line, with no extreme outliers. This demonstrates that the new offline signal acquisition protocol effectively eliminated interference issues from previous acquisitions to a certain extent.

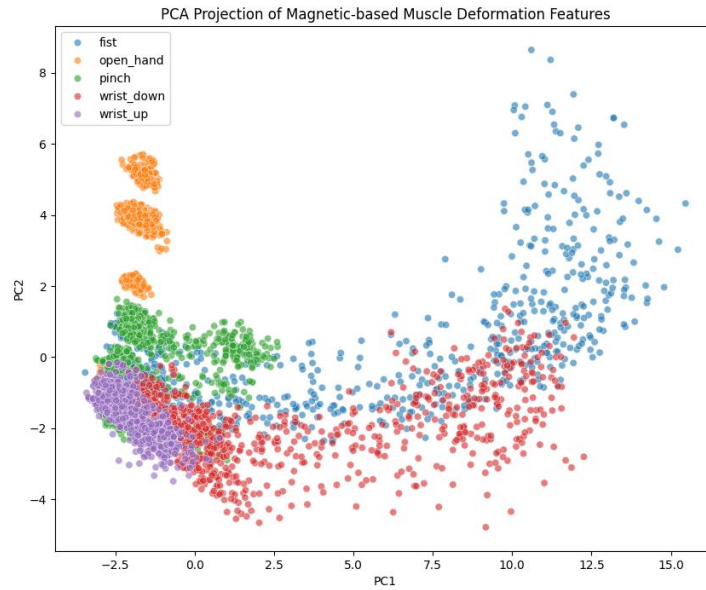


Figure 25. PCA projection of the final five gesture datasets acquired using a discrete data acquisition protocol. The data points now form tight, cohesive clusters with clear spatial separation

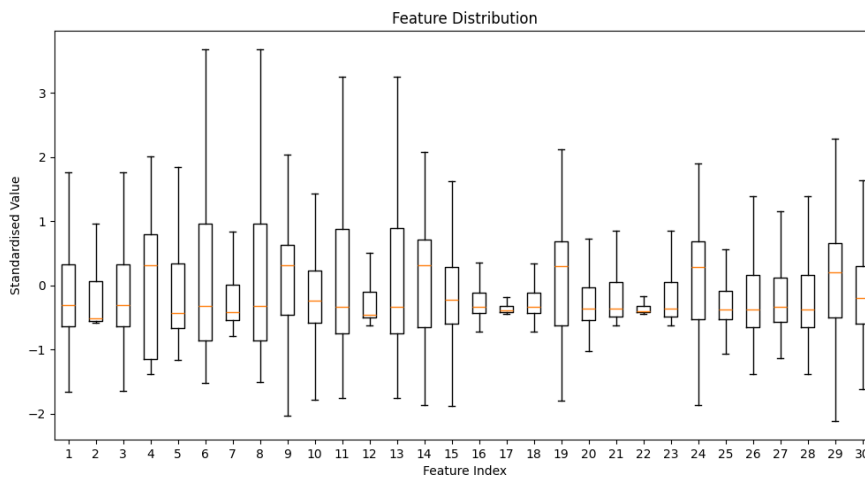


Figure 26. This is a boxplot of the normalised distribution of 30 time-domain features extracted from the dataset. No extreme outliers were found, confirming the stability of the baseline data before it was fed into the machine learning model

4.4. Offline algorithmic performance and the "Overfitting Illusion"

Among machine learning models, RF has consistently achieved the highest accuracy. Therefore, in subsequent detection tests, only RF and CNN were used for training and testing on the collected datasets. After training on the datasets collected using the latest acquisition protocol, RF achieved an impressive offline accuracy of 99.94%, compared to only around 70% for 1D-

CNN.

Figure 27 shows the RF confusion matrix, which classified the offline test data with almost zero errors. However, rather than proving the model is highly effective, this 99.94% score represents a well-documented "overfitting illusion" [48].

Random Forest (Full Dataset)

True Gesture	open_hand	1919	0	0	0	0
	fist	0	1988	0	0	0
	pinch	0	0	1905	0	0
	wrist_up	0	0	0	1935	0
	wrist_down	0	0	0	0	1941
		open_hand	fist	pinch	wrist_up	wrist_down
		Predicted Gesture				

Figure 27. Confusion matrix of the Random Forest classifier evaluated on the full static dataset. The zero off-diagonal values explicitly highlight the overfitting illusion

1D-CNN (Test Split)

True Gesture	open_hand	274	33	49	22	6
	fist	44	303	33	15	3
	pinch	90	31	212	44	4
	wrist_up	55	24	45	256	7
	wrist_down	12	16	14	16	330
		open_hand	fist	pinch	wrist_up	wrist_down
		Predicted Gesture				

Figure 28. Confusion matrix of the 1D-CNN model evaluated on the 20% hold-out test split, reflecting its true generalisation capacity and realistic classification errors

The core issue is data leakage. When a 75% overlapping sliding window is paired with random cross-validation, adjacent and nearly identical time windows get split across both the training and testing sets. Instead of learning the underlying muscle morphology, the algorithm simply

memorises the data points. Additionally, since the dataset was recorded during a single continuous session, the decision trees only memorised the absolute numerical baselines of that specific physical setup. Ultimately, this static offline test completely hides the RF model's failure to generalise against inter-session temporal drift.

4.5. Real-time dynamic performance, failure mode analysis, and safety-critical decisions

To further demonstrate this overfitting problem, the tests utilised the real-time prediction module on the GUI. By calling the trained model, gesture predictions were performed on real-time muscle deformation signals. Clearly, the previously mentioned data drift problem still exists in the machine learning model. Although the RF achieved near-perfect accuracy this time, it completely collapsed in real-time prediction. The model either got stuck on a single gesture or rapidly switched between several gestures, failing to effectively recognise them. Further analysis of the mathematical principles behind the time-domain features reveals why the model fails. Among the five features extracted earlier, indicators such as the mean and root mean square heavily rely on the absolute amplitude baseline. As discussed previously, the baseline changes to varying degrees whenever data collection spans different time periods or the wearing method changes. This is why real-time prediction fails.

Real-time prediction was then performed on the CNN. The results showed that although the prediction results still fluctuated, once stabilised at a gesture, the model could give a correct prediction (although, for example, the prediction accuracy for pinch was only 40-50%, this was still the highest prediction result given by the CNN for the current gesture). Moreover, it could recognise all the gestures collected during training, unlike previous machine learning models that were stuck on a certain gesture. The reason why the CNN can reduce the impact of this data drift to a certain extent is that a dynamic zero-mean normalisation operation (`window - np.mean(window, axis=0)`) was added to its neural network. This step can subtract the channel

mean in each window to ensure the relative shape of the waveform. This is why CNN can still make predictions even if objective factors still exist, even though the accuracy and stability are currently relatively low [21].

While normalised CNNs outperform machine learning algorithms for real-time gesture prediction, they also have issues that need improvement. For example, latency. Because the model needs to wait for all the data in the window to be collected before it can perform calculations and predictions, the size of the window determines the strength of the latency. This is why, during testing, the GUI window did not immediately display the prediction of the pinch gesture when the tester performed the pinch gesture; instead, there was a delay, and it flashed quickly (because the hand quickly returns to its natural state after the pinch gesture is completed). Secondly, current tests are all conducted via a GUI, i.e., on a computer. Data needs to be transmitted from the data acquisition unit to the computer, where the algorithm then processes it to obtain the results. This also increases latency to some extent. To achieve truly low-latency control, these algorithms need to be directly programmed into the embedded microcontroller [11].

To achieve safe manipulation of a realistic bionic hand, absolutely real-time control is essential [43]. Currently, only CNNs can perform predictions, but this still falls short of absolute control. Because CNNs still exhibit prediction jumps, deploying them on a dexterous hand could result in momentary servo malfunctions, potentially causing irreversible damage to the servos. Therefore, without further improvements in CNN weights and training, the model cannot be deployed on a dexterous hand for real-world testing.

5. Conclusions and Future Work

The project has successfully assembled and validated the core architecture of a low-cost, data-driven bionic hand. Although the final complete system was not deployed and tested, the bionic hand itself has completed motion testing, the signal acquisition unit can continuously collect datasets, the GUI can perfectly perform different tasks, and the algorithm can achieve preliminary gesture prediction.

The signal acquisition system underwent three major hardware iterations, a process that validated the principle that signal acquisition determines the limits of the entire project. If the data acquired at the front end is corrupted or contaminated, even the most complex and advanced classification algorithms cannot achieve accurate and effective predictions. The bionic hand, acquisition unit, and software algorithms are not independent modules; an error in any one link can cause the entire pipeline to collapse.

On the algorithmic model side, it became clear that training and testing solely on offline datasets does not guarantee the same accuracy in real-time testing. The extremely high accuracy achieved by machine learning models like RF in dataset testing is entirely due to data leakage caused by overlapping sliding windows. However, in real-time testing, changes in the baseline cause the model to completely fail. CNNs, by specifically incorporating dynamic zero-mean normalisation, exhibit significantly improved real-time prediction performance compared to RF. This demonstrates that with further adjustments to the weight parameters of CNNs, they are entirely feasible as a low-cost bionic hand control system.

To further achieve this goal, I believe the subsequent work can be broken down into the following steps and tackled one by one:

- **Mechanical Redesign for Thermal Management:** Squeezing seven servo motors into a sealed 3D-printed hand will inevitably lead to overheating during continuous use. Future mechanical improvements must re-examine the servo motor arrangement. Either

active cooling must be designed, or the actuator layout must be completely redesigned to achieve passive cooling. Alternatively, moving the servo motor array to a dedicated wrist or forearm module could reduce heat generation from overcrowding and free up physical space to increase the bionic hand's degrees of freedom.

- **Armband Structural Optimisation:** Current data-acquisition devices rely on rigid Velcro straps, which inevitably stretch and come loose during testing. Furthermore, these devices themselves lack ergonomic design. These issues can lead to even slight physical slippage, causing the sensors to shift and permanently disrupting the signal baseline. Future data-acquisition devices need to incorporate ergonomic designs that are comfortable to wear. This will ensure that the sensors remain in close contact with the skin without causing muscle fatigue. Only by capturing clear and complete signals can a solid foundation be laid for subsequent model training.
- **Dataset Scaling and Variance Injection:** Current offline datasets lack the large-scale data required to train deep learning models with genuine generalisation capabilities. Future data collection strategies will need to deliberately introduce environmental chaos. By recording data from different users over several days, under varying strap tensions and fluctuations in room temperature, this dataset will force neural networks to learn underlying biomechanical deformation patterns, rather than merely memorising the transient baseline states of the hardware.
- **Embedded Deployment and Edge Computing:** At present, the system is undergoing testing and data collection with the aid of a computer. Future practical deployment will require completely bypassing the computer, enabling direct data collection, training and control from the sensing units to the bionic hand. Once all preparatory work has been completed, these functions will need to be completely separated from the computer in order to achieve true commercialisation.

References

- [1] J. Vertongen, D. G. Kamper, G. Smit, and H. Vallery, ‘Mechanical Aspects of Robot Hands, Active Hand Orthoses, and Prostheses: A Comparative Review’, *IEEEASME Trans. Mechatron.*, vol. 26, no. 2, pp. 955–965, Apr. 2021, doi: 10.1109/TMECH.2020.3014182.
- [2] Y. Li, P. Wang, R. Li, M. Tao, Z. Liu, and H. Qiao, ‘A Survey of Multifingered Robotic Manipulation: Biological Results, Structural Evolvments, and Learning Methods’, *Front. Neurorobotics*, vol. 16, p. 843267, Apr. 2022, doi: 10.3389/fnbot.2022.843267.
- [3] A. Billard and D. Kragic, ‘Trends and challenges in robot manipulation’, *Science*, vol. 364, no. 6446, p. eaat8414, Jun. 2019, doi: 10.1126/science.aat8414.
- [4] N. Zhang *et al.*, ‘Biomimetic rigid-soft finger design for highly dexterous and adaptive robotic hands’, *Sci. Adv.*, vol. 11, no. 17, p. eadu2018, Apr. 2025, doi: 10.1126/sciadv.adu2018.
- [5] J. Ten Kate, G. Smit, and P. Breedveld, ‘3D-printed upper limb prostheses: a review’, *Disabil. Rehabil. Assist. Technol.*, vol. 12, no. 3, pp. 300–314, Apr. 2017, doi: 10.1080/17483107.2016.1253117.
- [6] O. M. Andrychowicz *et al.*, ‘Learning dexterous in-hand manipulation’, *Int. J. Robot. Res.*, vol. 39, no. 1, pp. 3–20, Jan. 2020, doi: 10.1177/0278364919887447.
- [7] F. Cordella *et al.*, ‘Literature Review on Needs of Upper Limb Prosthesis Users’, *Front. Neurosci.*, vol. 10, May 2016, doi: 10.3389/fnins.2016.00209.
- [8] Y.-P. Su, X.-Q. Chen, T. Zhou, C. Pretty, and G. Chase, ‘Mixed-Reality-Enhanced Human–Robot Interaction with an Imitation-Based Mapping Approach for Intuitive Teleoperation of a Robotic Arm-Hand System’, *Appl. Sci.*, vol. 12, no. 9, p. 4740, May 2022, doi: 10.3390/app12094740.
- [9] Y. Wan, J. Sun, C. Peers, J. Humphreys, D. Kanoulas, and C. Zhou, ‘Performance and Usability Evaluation Scheme for Mobile Manipulator Teleoperation’, *IEEE Trans. Hum.-Mach. Syst.*, vol. 53, no. 5, pp. 844–854, Oct. 2023, doi: 10.1109/THMS.2023.3289628.
- [10] M. Asghari Oskoei and H. Hu, ‘Myoelectric control systems—A survey’, *Biomed. Signal Process. Control*, vol. 2, no. 4, pp. 275–294, Oct. 2007, doi: 10.1016/j.bspc.2007.07.009.
- [11] R. Crepin, C. L. Fall, Q. Mascaret, C. Gosselin, A. Campeau-Lecours, and B. Gosselin, ‘Real-Time Hand Motion Recognition Using sEMG Patterns Classification’, in *2018 40th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, Honolulu, HI: IEEE, Jul. 2018, pp. 2655–2658. doi: 10.1109/EMBC.2018.8512820.
- [12] S. F. D. Toro, S. Santos-Cuadros, E. Olmeda, C. Álvarez-Caldas, V. Díaz, and J. L. San Román, ‘Is the Use of a Low-Cost sEMG Sensor Valid to Measure Muscle Fatigue?’, *Sensors*, vol. 19, no. 14, p. 3204, Jul. 2019, doi: 10.3390/s19143204.
- [13] S. Pizzolato, L. Tagliapietra, M. Cognolato, M. Reggiani, H. Müller, and M. Atzori,

- ‘Comparison of six electromyography acquisition setups on hand movement classification tasks’, *PLOS ONE*, vol. 12, no. 10, p. e0186132, Oct. 2017, doi: 10.1371/journal.pone.0186132.
- [14] J. I. Efanov, B. Tchiloemba, A. Izadpanah, P. G. Harris, and M. A. Danino, ‘A review of utilities and costs of treating upper extremity amputations with vascularized composite allotransplantation versus myoelectric prostheses in Canada’, *JPRAS Open*, vol. 32, pp. 150–160, Jun. 2022, doi: 10.1016/j.jpra.2022.03.003.
- [15] K. Guo, J. Lu, Y. Wu, X. Hu, and H. Yang, ‘The Latest Research Progress on Bionic Artificial Hands: A Systematic Review’, *Micromachines*, vol. 15, no. 7, p. 891, Jul. 2024, doi: 10.3390/mi15070891.
- [16] J. J. A. Mendes Junior *et al.*, ‘AD8232 to Biopotentials Sensors: Open Source Project and Benchmark’, *Electronics*, vol. 12, no. 4, p. 833, Feb. 2023, doi: 10.3390/electronics12040833.
- [17] J. S. Karlsson, K. Roeleveld, C. Grönlund, A. Holtermann, and N. Östlund, ‘Signal processing of the surface electromyogram to gain insight into neuromuscular physiology’, *Philos. Trans. R. Soc. Math. Phys. Eng. Sci.*, vol. 367, no. 1887, pp. 337–356, Jan. 2009, doi: 10.1098/rsta.2008.0214.
- [18] J. M. Hahne *et al.*, ‘Linear and Nonlinear Regression Techniques for Simultaneous and Proportional Myoelectric Control’, *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 22, no. 2, pp. 269–279, Mar. 2014, doi: 10.1109/TNSRE.2014.2305520.
- [19] G. Li, S. Wang, and Y. Y. Duan, ‘Towards gel-free electrodes: A systematic study of electrode-skin impedance’, *Sens. Actuators B Chem.*, vol. 241, pp. 1244–1255, Mar. 2017, doi: 10.1016/j.snb.2016.10.005.
- [20] E. Scheme and K. Englehart, ‘Electromyogram pattern recognition for control of powered upper-limb prostheses: State of the art and challenges for clinical use’, *J. Rehabil. Res. Dev.*, vol. 48, no. 6, p. 643, 2011, doi: 10.1682/JRRD.2010.09.0177.
- [21] U. Cote-Allard *et al.*, ‘Deep Learning for Electromyographic Hand Gesture Signal Classification Using Transfer Learning’, *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 27, no. 4, pp. 760–771, Apr. 2019, doi: 10.1109/TNSRE.2019.2896269.
- [22] L. Dipietro, A. M. Sabatini, and P. Dario, ‘A Survey of Glove-Based Systems and Their Applications’, *IEEE Trans. Syst. Man Cybern. Part C Appl. Rev.*, vol. 38, no. 4, pp. 461–482, Jul. 2008, doi: 10.1109/TSMCC.2008.923862.
- [23] V. Ravindra and C. Castellini, ‘A Comparative Analysis of Three Non-Invasive Human-Machine Interfaces for the Disabled’, *Front. Neurobotics*, vol. 8, Oct. 2014, doi: 10.3389/fnbot.2014.00024.
- [24] A. Radmand, E. Scheme, and K. Englehart, ‘High-density force myography: A possible alternative for upper-limb prosthetic control’, *J. Rehabil. Res. Dev.*, vol. 53, no. 4, pp. 443–456, 2016, doi: 10.1682/JRRD.2015.03.0041.

- [25] A. Belyea, K. Englehart, and E. Scheme, 'FMG Versus EMG: A Comparison of Usability for Real-Time Pattern Recognition Based Control', *IEEE Trans. Biomed. Eng.*, vol. 66, no. 11, pp. 3098–3104, Nov. 2019, doi: 10.1109/TBME.2019.2900415.
- [26] C. S. Lovchik and M. A. Diftler, 'The Robonaut hand: a dexterous robot hand for space', in *Proceedings 1999 IEEE International Conference on Robotics and Automation (Cat. No.99CH36288C)*, Detroit, MI, USA: IEEE, 1999, pp. 907–912. doi: 10.1109/ROBOT.1999.772420.
- [27] 'Optimus Hand Design', tesla.rocks. Accessed: Oct. 05, 2025. [Online]. Available: <https://tesla.rocks/2025/06/01/optimus-hand-design/>
- [28] 'All You Need to Know About the Unitree Dex5 Hand', humanoid.guide. Accessed: Oct. 05, 2025. [Online]. Available: <https://humanoid.guide/all-you-need-to-know-about-the-unitree-dex5-hand/#:~:text=Handling%20soft%20fruits%20like%20oranges,simple%20games%20and%20magic%20tricks!>
- [29] S. Marco, P. Nicola, G. Paolo, and P. Giovanni, 'Virtual and Rapid Prototyping of an Underactuated Space end Effector', *J. Robot. Autom.*, vol. 1, no. 1, Apr. 2017, doi: 10.36959/673/351.
- [30] Y. Xu and Z. Jiang, 'Energy Efficiency and Force Transmission Optimization in Modular Robotic Arm Design Configuration', in *2025 IEEE 3rd International Conference on Image Processing and Computer Applications (ICIPCA)*, Shenyang, China: IEEE, Jun. 2025, pp. 1–5. doi: 10.1109/ICIPCA65645.2025.11139024.
- [31] X. Zhou, H. Fu, B. Shentu, W. Wang, S. Cai, and G. Bao, 'Design and Control of a Tendon-Driven Robotic Finger Based on Grasping Task Analysis', *Biomimetics*, vol. 9, no. 6, p. 370, Jun. 2024, doi: 10.3390/biomimetics9060370.
- [32] Y. Zhang, Q. Zhao, H. Deng, and X. Xu, 'Design of a Multi-Mode Mechanical Finger Based on Linkage and Tendon Fusion Transmission', *Biomimetics*, vol. 8, no. 3, p. 316, Jul. 2023, doi: 10.3390/biomimetics8030316.
- [33] R. Merletti and P. J. Parker, *Electromyography: Physiology, Engineering, and Non-Invasive Applications*. John Wiley & Sons, 2004.
- [34] D. Farina *et al.*, 'The Extraction of Neural Information from the Surface EMG for the Control of Upper-Limb Prostheses: Emerging Avenues and Challenges', *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 22, no. 4, pp. 797–809, Jul. 2014, doi: 10.1109/TNSRE.2014.2305111.
- [35] A. Phinyomark, P. Phukpattaranont, and C. Limsakul, 'Feature reduction and selection for EMG signal classification', *Expert Syst. Appl.*, vol. 39, no. 8, pp. 7420–7431, Jun. 2012, doi: 10.1016/j.eswa.2012.01.102.
- [36] A. H. Al-Timemy, G. Bugmann, J. Escudero, and N. Outram, 'Classification of Finger Movements for the Dexterous Hand Prosthesis Control With Surface Electromyography',

- IEEE J. Biomed. Health Inform.*, vol. 17, no. 3, pp. 608–618, May 2013, doi: 10.1109/JBHI.2013.2249590.
- [37] Ning Jiang, S. Dosen, K.-R. Muller, and D. Farina, ‘Myoelectric Control of Artificial Limbs—Is There a Need to Change Focus? [In the Spotlight]’, *IEEE Signal Process. Mag.*, vol. 29, no. 5, pp. 152–150, Sep. 2012, doi: 10.1109/MSP.2012.2203480.
- [38] Y.-W. Chao *et al.*, ‘DexYCB: A Benchmark for Capturing Hand Grasping of Objects’, Apr. 09, 2021, *arXiv*: arXiv:2104.04631. doi: 10.48550/arXiv.2104.04631.
- [39] ‘Compare Gloves Specifications | MANUS’. [Online]. Available: <https://www.manus-meta.com/products/compare-gloves>
- [40] C. Lugaresi *et al.*, ‘MediaPipe: A Framework for Building Perception Pipelines’, Jun. 14, 2019, *arXiv*: arXiv:1906.08172. doi: 10.48550/arXiv.1906.08172.
- [41] C. Zimmermann and T. Brox, ‘Learning to Estimate 3D Hand Pose from Single RGB Images’, 2017, *arXiv*. doi: 10.48550/ARXIV.1705.01389.
- [42] M. Cognolato *et al.*, ‘Gaze, visual, myoelectric, and inertial data of grasps for intelligent prosthetics’, *Sci. Data*, vol. 7, no. 1, p. 43, Feb. 2020, doi: 10.1038/s41597-020-0380-3.
- [43] K. Englehart and B. Hudgins, ‘A robust, real-time control scheme for multifunction myoelectric control’, *IEEE Trans. Biomed. Eng.*, vol. 50, no. 7, pp. 848–854, Jul. 2003, doi: 10.1109/TBME.2003.813539.
- [44] F. Zhang *et al.*, ‘MediaPipe Hands: On-device Real-time Hand Tracking’, Jun. 18, 2020, *arXiv*: arXiv:2006.10214. doi: 10.48550/arXiv.2006.10214.
- [45] *TetherIA/aero-hand-open*. Python. TetherIA. [Online]. Available: <https://github.com/TetherIA/aero-hand-open>
- [46] D. P. Kingma and J. Ba, ‘Adam: A Method for Stochastic Optimization’, Jan. 30, 2017, *arXiv*: arXiv:1412.6980. doi: 10.48550/arXiv.1412.6980.
- [47] L. Breiman, ‘Random Forests’, *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324.
- [48] N. Y. Hammerla and T. Plötz, ‘Let’s (not) stick together: pairwise similarity biases cross-validation in activity recognition’, in *Proceedings of the 2015 ACM International Joint Conference on Pervasive and Ubiquitous Computing*, Osaka Japan: ACM, Sep. 2015, pp. 1041–1051. doi: 10.1145/2750858.2807551.

Appendix A

Table 6. Bill of materials for TetherIA right hand and acquisition units

Categories	Item Name	Number	unit price(RMB)	Total
sensors	AD8232 Sensor kit	2	69.6	139.2
	SICHIRAY sEMG sets	2	95	190
	ESP32 Development board	1	20	20
	DuPont wire bundle	1	12	12
	Micro-USB Data cable	1	8.61	8.61
	Breadboard(400)	2	15	30
	Disposable electrode patch	1	28.8	28.8
	sEMG dry electrode single-channel electromyography sensor	2	49	98
	Muscle Deformation Armband	1	599	599
Tether	Finger Distal/Medial and Thumb Proximal/Distal Return Spring (PIP)	5	2.88	14.4
	Finger Proximal/MCP Return Spring (MP)	4	3.6	14.4
	Thumb CMC/MCP Return Spring (CMC)	1	5.4	5.4
	Wear-resistant and cut-resistant polymer rope <=1.2mm	1	30.8	30.8
	Finger Pull Cable	3		0
	Pinky Pull Cable	1		0
	Finger Coupling Cable	4		0
	Thumb CMC Flex Cable	1		0
	Thumb Pull Cable	1		0
	Thumb Coupling Cable	1		0
	2x5x2.5 bearing	21	1.5	31.5
	1x10 Pin	16		0
	obsolete (2x10 Pin)	0		0
	2x14 Pin	12		0
	2x20 Pin	2		0
	UNDERSIZED 2x10 Pin	22		0
	UNDERSIZED 2x20 Pin	1		0
	UNDERSIZED 2x30 Pin	1		0
	M2x6 Flanged Torx Self Tapping Screw	25		0
	Tapping Screw	12		0
	M3x10 Rounded Torx Self Tapping Screw	9		0
	M3x5.7 Heat set insert	4	0.4075	1.63

A set of servos	Feetech HLS3606M Servo Motor	7	345.15	2416.05
	Servo Motor Mount Plate			0
	Servo Motor Spline Screw M2.3x5			0
	Servo Motor Plate Screw M2x4			0
	Connection Board and Cable			0
	Conversion Board			0
	ESP32-S3	1	33.4	33.4
	Power adapter	1	52	52
			Total(RMB)	3725.19

Table 7. Failure Mode, Effects, and Criticality Analysis (FMECA) for TetherIA right hand

Failure Modes Effects and Critical Analysis (FMECA)								
Part: TetherIA Aero Hand Open			Version: 1st version		Lihao ZHOU		Date: 05/01/2026	
Item / Component	Failure Mode	Failure Cause	Failure Effect	Severity (S)	Occurrence (O)	Detection (D)	RPN	Recommended Mitigation Action
Tendon Cables (Tension Lines)	Mechanical snapping or severe plastic deformation	High friction within routing channels; excessive pulling force from servos	Loss of actuation in the affected digit; severe mechanical hysteresis	8	8	4	256	Utilise high-strength UHMWPE lines; apply PTFE lubricant to internal routing paths.
Actuators (Servo Motors)	Gear stripping or thermal overload (burnout)	Continuous stall current when physical movement is blocked; internal friction	Permanent loss of motor function; potential electrical short circuit	9	5	5	225	Implement software-level angular constraints; upgrade to metal-gear servos.
Tendon Routing Channels	Tendon jamming / excessive dynamic friction	Rough internal surfaces from 3D printed support material; acute bending angles	Delayed finger response; increased electrical load and heat on servos	6	9	3	162	Conduct mechanical post-processing (sanding) of internal channels; minimise sharp routing angles.
Structural Fasteners (Pins/Screws)	Loosening or mechanical shear	Cyclic dynamic loading during continuous grasping cycles	Joint instability; imprecise kinematic output at the fingertips	6	5	4	120	Apply thread-locking fluid during assembly; utilise metal pins for primary rotary joints.
3D-Printed Phalanges (Finger Joints)	Structural fracture or layer delamination	Mechanical stress from tendon tension; poor interlayer adhesion during FDM printing	Complete loss of structural integrity; inability to perform flexion/extension	8	6	2	96	Increase 3D print infill density; optimise print orientation for load distribution.

Table 8. Rating system for FMECA

Rating	Severity (S) - Impact on System Operation	Occurrence (O) - Probability of Failure	Detection (D) - Likelihood of Detecting Before Failure
10	Hazardous: Complete system failure; potential safety hazard.	Inevitable: Failure is almost certain to occur repeatedly.	Absolute Uncertainty: No method exists to detect the failure before it occurs.
9	Critical: Primary function lost; requires complete hardware replacement.	Very High: Failures occur frequently during normal operation.	Very Low: Detection is highly improbable before catastrophic failure.
8	High: Severe degradation of primary function; significant mechanical stall.	High: Repeated failures observed during testing phases.	Low: System has minimal capacity to detect the impending issue.
7	Major: Partial loss of primary function; high execution hysteresis.	Moderately High: Failures occur occasionally.	Moderately Low: Detection requires specific, non-routine manual inspection.
6	Significant: System operates but with noticeable physical degradation.	Moderate: Occasional failures, but statistically manageable.	Moderate: Issue can be detected during routine system calibration.
5	Moderate: Minor loss of performance; necessitates hardware recalibration.	Occasional: Isolated incidents; not a persistent pattern.	Fair: Moderate probability of detection via software anomalies or physical visual checks.
4	Minor: System functions with slight friction or delay; acceptable tolerance.	Low: Few failures expected during the component's lifespan.	Moderately High: High probability of detection during standard pre-flight checks.
3	Slight: Negligible impact on overall data acquisition or kinematics.	Very Low: Failures are rare and highly unlikely.	High: Very likely to be detected immediately by system feedback.
2	Very Slight: Barely noticeable effect; cosmetic or transient issue.	Remote: Extremely unlikely to occur under normal parameters.	Very High: Nearly certain to be detected by built-in continuous monitoring.
1	None: No discernible impact on system functionality.	Improbable: History of no failures for similar engineering designs.	Certain: Automated and immediate detection guaranteed prior to operational failure.

Appendix B

Standardised Hand Gesture Data Collection Protocol

This protocol defines the exact physical steps and constraints required during the kinematic and sensor data collection phase. The resulting dataset provides the training and testing boundaries to evaluate the pattern recognition algorithms under low-cost hardware limitations.

B.1. Global Operational Constraints

To protect data integrity, remove transient muscle artifacts, and prevent signal baseline drift, the user must strictly follow these three operational rules throughout the entire recording session:

- **Hardware Pre-load Standardisation:** Do not touch, adjust, or rotate the magnetic muscle deformation armband once the baseline is locked. Every single gesture cycle must share the exact same physical sensor alignment.
- **Mechanical Hysteresis Elimination:** After each 2.5-second recording window closes, the subject must completely relax the forearm muscles and pause for 1 to 2 seconds before hitting the spacebar for the next trial. This drop-to-zero pause resets the physical tissue structure.
- **Dataset Matrix Balancing:** Keep a strict count of 50 discrete repetitions for every single gesture class. Maintaining perfect parity across all categories ensures a balanced training matrix for the machine learning pipeline.

B.2. Systematic Execution Sequence

Data collection follows a strict order, moving from a completely passive baseline to macroscopic wrist movements, and finally to highly localised finger deformations.

B.2.1. Gesture Class 1: Natural Resting State (Passive Open Hand)

- **Objective:** Record the un-contracted muscle profile and measure the baseline noise floor of the Hall-effect sensor array.

- **Protocol:** Lay the forearm flat on a horizontal desk with the hand naturally open and completely relaxed. Record pure baseline values with zero voluntary muscle activation.

B.2.2. Gesture Class 2: Power Grasp (Fist)

- **Objective:** Map the maximum inward physical displacement and contraction limits of the flexor muscle groups.
- **Reps 1–15 (Standard Baseline):** Keep the forearm flat on the desk and actively clench all five fingers into a tight, solid fist.
- **Reps 16–30 (Pose Crosstalk):** Lift the forearm and suspend it fully in mid-air. Twist and rotate the arm angle across repetitions while holding the fist tight.
- **Reps 31–45 (Force Gradient):** Divide trials evenly between minimal closures (fingers just touching the palm with no force), standard functional grips, and maximum-effort rigid squeezes.
- **Reps 46–50 (Transient Response):** Trigger explosive, instantaneous fists snapped rapidly from a completely relaxed resting state.

B.2.3. Gesture Class 3: Precision Pinch

- **Objective:** Capture the highly localised shape changes of minor muscle contractions that naturally suffer from a low signal-to-noise ratio (SNR).
- **Operational Constraint:** The subject must exclusively use the thumb paired with either the middle or ring finger to execute the pinch.
- **Reps 1–15 (Standard Baseline):** Keep the forearm flat on the desk and execute a precise, standard oppositional pinch.
- **Reps 16–30 (Isolated Control):** Maintain the pinch contact while intentionally altering the posture of the non-participating fingers (e.g., index finger fully extended versus tightly curled into the palm).
- **Reps 31–45 (Force Gradient):** Distribute trials evenly between superficial skin contact

(pads touching with minimal pressure), normal operational pinches, and maximum-force pinches.

- **Reps 46–50 (Anti-Gravity Crosstalk):** Suspend the forearm fully in the air to intentionally inject mechanical crosstalk from the proximal shoulder and arm-stabilising muscle groups.

B.2.4. Gesture Class 4: Wrist Flexion (Wrist Down)

- **Objective:** Record the primary macroscopic displacement of the flexor carpi radialis and main forearm flexor groups during inward wrist movements.
- **Reps 1–15 (Standard Baseline):** Lay the forearm flat on the desk with the wrist suspended over the edge. Actively flex the palm downwards and inwards towards the body.
- **Reps 16–30 (Digit Crosstalk):** Alternating between flexions done with the fingers fully extended and flexions done while holding a loose fist. This tests the structural superposition of the finger and wrist flexors.
- **Reps 31–45 (Force Gradient):** Distribute trials evenly between minimal downward tilts, standard structural flexions, and maximum-angle bends.
- **Reps 46–50 (Suspended Dynamic):** Execute the full downward flexion path with the entire arm fully suspended in mid-air.

B.2.5. Gesture Class 5: Wrist Extension (Wrist Up)

- **Objective:** Record the primary macroscopic displacement of the extensor carpi radialis longus and forearm extensor groups during upward wrist movements.
- **Reps 1–15 (Standard Baseline):** Lay the forearm flat on the desk with the wrist suspended over the edge. Actively extend the back of the hand upward towards the forearm.
- **Reps 16–30 (Deflection Crosstalk):** Run the extensions while simultaneously adding

slight left or right deviations (radial/ulnar shifts) to inject lateral tracking crosstalk.

- **Reps 31–45 (Force Gradient):** Distribute trials evenly between minimal vertical lifts, normal extensions, and maximum-angle structural lifts.
- **Reps 46–50 (Suspended Dynamic):** Execute the full upward extension path with the entire arm fully suspended in space.

Appendix C

Algorithm 1: Real-time Data Acquisition and Multithreaded GUI Pipeline

Inputs: COM_Port (115200 Baud), Sampling_Rate (31Hz)

Outputs: Space-separated TXT Dataset Files

```
-----
1: Initialize Thread-safe Data_Queue, Record_Flag = False
2:
3: // Thread 1: Serial Receiver (Worker Thread)
4: Function SerialReceiverThread():
5:   While connection is open:
6:     Raw_Data <- Read serial line
7:     If Raw_Data is valid:
8:       Push Parsed_Values[6] to Data_Queue
9:
10: // Thread 2: UI Rendering and Logging (Main Thread)
11: Function UIRenderLoop():
12:   While UI_Window is active:
13:     If Data_Queue is not empty:
14:       Current_Values <- Pop from Data_Queue
15:       Update UI_Plot(Current_Values)
16:
17:     If Record_Flag is True:
18:       Append Current_Values to Record_Buffer
19:       Wait (1/Sampling_Rate)
20:
21: // Timer Callback (Triggered 2.5s after Spacebar)
22: Function StopRecording(Target_Gesture):
23:   Record_Flag <- False
24:   Filename <- "{Target_Gesture}_raw_{Timestamp}.txt"
25:   Save Record_Buffer to Filename (Space-separated format)
26:   Clear Record_Buffer
27:
28: // Event Listener: Keystroke Trigger
29: Function OnKeyPressEvent(Key):
30:   If Key == Spacebar AND Record_Flag is False:
31:     Record_Flag <- True
```

32: Start Timer(2.5 seconds, StopRecording)
33: Start SerialReceiverThread() AND UIRenderLoop()